

Task 4 — Per-Round LLM Iteration Trace (weak_gpt4o_mini)

Author: Zimo Ji

Per-round dump of the prompts I sent and the responses I received from `gpt-4o-mini` while iterating the test-driven code-generation experiment for `NewSubject.java`. Files reproduced verbatim from `reports/Task4_Data/weak_gpt4o_mini/{prompts,llm_outputs,test_results}/`.

Model: `gpt-4o-mini`, temperature 0.1, accessed via an OpenAI-compatible chat-completions endpoint.

Round-by-round summary

round	compile_ok	run	fail	pass_rate
1	True	62	2	96.8%
2	True	62	2	96.8%
3	True	62	2	96.8%
4	True	62	2	96.8%
5	True	62	2	96.8%

Round 1

Test outcome

```
{
  "compile_ok": true,
  "run_count": 62,
  "fail_count": 2,
  "failures": [
    "test51(comp5111.assignment.cut.NewSubject_ESTest) :: null",
    "test43(comp5111.assignment.cut.NewSubject_ESTest) :: expected:<29> but was:<28>"
  ],
  "error": "",
  "raw_stdout": "10:33:11.451 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject_ESTest\n10:33:11.453 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject_ESTest'.\n10:33:11.480 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject_ESTest_scaffolding\n10:33:11.480 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject_ESTest_scaffolding'.\n10:33:11.482 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject_ESTest_scaffolding\n10:33:11.482 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject_ESTest\n10:33:11.659 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread: \"Reference
```

```

Handler\"n10:33:11.659 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Finalizer\"n10:33:11.659 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread: \"Signal
Dispatcher\"n10:33:11.659 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Java2D Disposer\"n10:33:11.659 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread:
\"main\"n10:33:11.659 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Common-Cleaner\"n10:33:11.677 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubjectn10:33:11.677 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject'.n10:33:11.682 [main] INFO
org.evosuite.runtime.instrumentation.MethodCallReplacementClassAdapter - Adding mock
interface to class comp5111/assignment/cut/NewSubjectn10:33:11.753 [main] INFO
org.evosuite.runtime.instrumentation.CreateClassResetClassAdapter - Creating brand-new
static initializer in class comp5111/assignment/cut/NewSubjectn10:33:11.754 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubjectn10:33:11.772 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new threadn10:33:11.777 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new threadn10:33:11.785 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new threadn10:33:11.789 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new threadn10:33:11.794 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new threadn10:33:11.795 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new threadn10:33:11.796 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new threadn10:33:11.797 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new threadn10:33:11.798 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new threadn10:33:11.800 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new threadn10:33:11.801 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new
threadnRUN_COUNT=62nFAIL_COUNT=2nFAILURE>>>test51(comp5111.assignment.cut.NewSubject_EST
null<<<FAILUREnFAILURE>>>test43(comp5111.assignment.cut.NewSubject_ESTest) ::
expected:<29> but was:<28><<<FAILUREn\",
  \"raw_stderr\": \"WARNING: An illegal reflective access operation has occurrednWARNING:
Illegal reflective access by org.evosuite.runtime.GuiSupport (file:/home/zimo/playground/
COMP5111-2026Spring-Assignments-Students/lib/evosuite-1.2.0.jar) to field
java.awt.GraphicsEnvironment.headlessnWARNING: Please consider reporting this to the
maintainers of org.evosuite.runtime.GuiSupportnWARNING: Use --illegal-access=warn to
enable warnings of further illegal reflective access operationsnWARNING: All illegal
access operations will be denied in a future release\n\"
}

```

Prompt sent to LLM (15138 chars)

You are a Java engineer. Implement the class `comp5111.assignment.cut.NewSubject` so that every JUnit test in the provided test suite passes.

Rules:

- Output exactly ONE fenced ``java code block containing the FULL contents of NewSubject.java (package, imports if any, the entire class).
- Do NOT modify the test suite. Do NOT add main(). Keep the package and class name unchanged. Method signatures shown below are mandatory.
- Output nothing outside the single code block.

=== Method signatures + Javadoc (mandatory shape) ===

```
package comp5111.assignment.cut;
```

```
/**
```

```
 * Task 4 reduced CUT.
```

```
 *
```

```
 * Ten public-static methods selected from {@link Subject}, with their
```

```

* documented contracts and one private helper (isOneOf). The patched
* (correct) implementations from Task 2 are used so that EvoSuite's
* generated tests pin down the *intended* behaviour, against which the
* LLM's regenerated code is judged.
*
* Every signature is identical to the corresponding method in Subject.
*/
public class NewSubject {

    // ---- 1 ----
    /**
     * Returns true if str starts with prefix ignoring case, else false.
     * Returns false on any null argument or if str is shorter than prefix.
     */
    public static boolean startsWithIgnoreCase(String str, String prefix);

    // ---- 2 ----
    /**
     * Returns the prefix of str up to (but not including) the first
     * character that appears in {@code terminators}. If no terminator
     * is found, the whole string is returned.
     */
    public static String parseToken(final String str, final char[] terminators);

    private static boolean isOneOf(char ch, final char[] chararray);

    // ---- 3 ----
    /**
     * Returns the LAST contiguous run of digits in str interpreted as
     * an int. Empty/null/no-digit inputs return 0.
     * Examples: "1234" -> 1234, "a" -> 0, "1234a123" -> 123.
     */
    public static int extractIntInStr(String str);

    // ---- 4 ----
    /**
     * Parses a version string of form a, a.b, a.b.c, or a.b.c.d into a 4-int
     * array. Trailing components default to 0. More than 4 dot-segments or
     * malformed (consecutive dots) produces null.
     */
    public static int[] getVersionNo(final String versionString);

    // ---- 5 ----
    /**
     * Pads str on the left with padChar so the resulting length is at least
     * length. Null str is treated as "". If str is already long enough it is
     * returned unchanged.
     */
    public static String padLeft(String str, short length, char padChar);

    // ---- 6 ----
    /** Returns true iff year is a leap year (Gregorian rule). */
    public static boolean judgeLeapYear(int year);

    // ---- 7 ----
    /**
     * Returns the number of days in the given month (1-12) of the given
     * year. Returns -1 if month is out of range. February uses leap-year
     * rules.
     */
    public static int calcDaysInMonth(int year, int month);

    // ---- 8 ----
    /** Returns the calendar quarter (1-4) for month (1-12), or -1 if invalid. */

```

```

    public static int getQuarter(int month);

    // ---- 9 ----
    /**
     * Converts a 3-char month abbreviation (case-sensitive, e.g. "Jan",
     * "Sep") into its month number (1-12). Returns -1 on null, wrong
     * length, or unknown abbreviation.
     */
    public static int monAbbr2month(String abbr);

    // ---- 10 ----
    /**
     * Inverse of monAbbr2month: returns "Jan".. "Dec" for month 1..12, and
     * "(invalid)" otherwise.
     */
    public static String month2MonAbbr(int month);
}

```

=== EvoSuite-generated test suite (passes against the ground truth) ===

```

/*
 * This file was automatically generated by EvoSuite
 * Thu May 07 11:32:05 GMT 2026
 */

package comp5111.assignment.cut;

import org.junit.Test;
import static org.junit.Assert.*;
import comp5111.assignment.cut.NewSubject;
import org.evosuite.runtime.EvoRunner;
import org.evosuite.runtime.EvoRunnerParameters;
import org.junit.runner.RunWith;

@RunWith(EvoRunner.class) @EvoRunnerParameters(mockJVMNonDeterminism = true, useVFS =
true, useVNET = true, resetStaticState = true, separateClassLoader = true)
public class NewSubject_ESTest extends NewSubject_ESTest_scaffolding {

    @Test(timeout = 4000)
    public void test00() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(1493);
        assertEquals("(invalid)", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test01() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(12);
        assertEquals("Dec", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test02() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(11);
        assertEquals("Nov", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test03() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(10);
        assertEquals("Oct", string0);
    }
}

```

```

        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test04() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(9);
        assertEquals("Sep", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test05() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(8);
        assertEquals("Aug", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test06() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(7);
        assertEquals("Jul", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test07() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(6);
        assertEquals("Jun", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test08() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(5);
        assertEquals("May", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test09() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(4);
        assertEquals("Apr", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test10() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(3);
        assertEquals("Mar", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test11() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(2);
        assertEquals("Feb", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test12() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(1);
        assertEquals("Jan", string0);
        assertNotNull(string0);
    }

```

```

}

@Test(timeout = 4000)
public void test13() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Dec");
    assertEquals(12, int0);
}

@Test(timeout = 4000)
public void test14() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Nov");
    assertEquals(11, int0);
}

@Test(timeout = 4000)
public void test15() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Oct");
    assertEquals(10, int0);
}

@Test(timeout = 4000)
public void test16() throws Throwable {
    int int0 = NewSubject.monAbbr2month("'Y~");
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test17() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Aug");
    assertEquals(8, int0);
}

@Test(timeout = 4000)
public void test18() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Jul");
    assertEquals(7, int0);
}

@Test(timeout = 4000)
public void test19() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Jun");
    assertEquals(6, int0);
}

@Test(timeout = 4000)
public void test20() throws Throwable {
    int int0 = NewSubject.monAbbr2month("May");
    assertEquals(5, int0);
}

@Test(timeout = 4000)
public void test21() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Apr");
    assertEquals(4, int0);
}

@Test(timeout = 4000)
public void test22() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Mar");
    assertEquals(3, int0);
}

@Test(timeout = 4000)
public void test23() throws Throwable {

```

```

        int int0 = NewSubject.monAbbr2month("Sep");
        assertEquals(9, int0);
    }

    @Test(timeout = 4000)
    public void test24() throws Throwable {
        int int0 = NewSubject.monAbbr2month("Jan");
        assertEquals(1, int0);
    }

    @Test(timeout = 4000)
    public void test25() throws Throwable {
        int int0 = NewSubject.monAbbr2month("(invalid)");
        assertEquals((-1), int0);
    }

    @Test(timeout = 4000)
    public void test26() throws Throwable {
        int int0 = NewSubject.monAbbr2month("Feb");
        assertEquals(2, int0);
    }

    @Test(timeout = 4000)
    public void test27() throws Throwable {
        int int0 = NewSubject.monAbbr2month((String) null);
        assertEquals((-1), int0);
    }

    @Test(timeout = 4000)
    public void test28() throws Throwable {
        int int0 = NewSubject.getQuarter(11);
        assertEquals(4, int0);
    }

    @Test(timeout = 4000)
    public void test29() throws Throwable {
        int int0 = NewSubject.getQuarter(4);
        assertEquals(2, int0);
    }

    @Test(timeout = 4000)
    public void test30() throws Throwable {
        int int0 = NewSubject.getQuarter((short)8);
        assertEquals(3, int0);
    }

    @Test(timeout = 4000)
    public void test31() throws Throwable {
        int int0 = NewSubject.getQuarter(946);
        assertEquals((-1), int0);
    }

    @Test(timeout = 4000)
    public void test32() throws Throwable {
        int int0 = NewSubject.getQuarter(2);
        assertEquals(1, int0);
    }

    @Test(timeout = 4000)
    public void test33() throws Throwable {
        int int0 = NewSubject.getQuarter((-1460));
        assertEquals((-1), int0);
    }
}

```

```

@Test(timeout = 4000)
public void test34() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth(11, 11);
    assertEquals(30, int0);
}

@Test(timeout = 4000)
public void test35() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth((-1), 9);
    assertEquals(30, int0);
}

@Test(timeout = 4000)
public void test36() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth(0, 6);
    assertEquals(30, int0);
}

@Test(timeout = 4000)
public void test37() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth(2, 4);
    assertEquals(30, int0);
}

@Test(timeout = 4000)
public void test38() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth((-1), 2);
    assertEquals(28, int0);
}

@Test(timeout = 4000)
public void test39() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth(0, 3);
    assertEquals(31, int0);
}

@Test(timeout = 4000)
public void test40() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth(1, 1726);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test41() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth((-3324), (-1570));
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test42() throws Throwable {
    boolean boolean0 = NewSubject.judgeLeapYear(3300);
    assertFalse(boolean0);
}

@Test(timeout = 4000)
public void test43() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth((-3324), 2);
    assertEquals(29, int0);
}

@Test(timeout = 4000)
public void test44() throws Throwable {
    boolean boolean0 = NewSubject.judgeLeapYear(0);
    assertTrue(boolean0);
}

```



```

}

@Test(timeout = 4000)
public void test45() throws Throwable {
    String string0 = NewSubject.padLeft("Jun", (short)1741, 'G');
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test46() throws Throwable {
    String string0 = NewSubject.padLeft((String) null, (short) (-1), '(');
    assertEquals("", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test47() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo(".@:r~w'iW");
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test48() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo("oZ.>B6oZ.>B6oZ.>B6oZ.>B6");
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test49() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo("");
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test50() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo((String) null);
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test51() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo("$S3P&,*V+D&1{%U6");
    assertEquals(4, intArray0.length);
    assertNotNull(intArray0);
    assertEquals(new int[] {6, 0, 0, 0}, intArray0);
}

@Test(timeout = 4000)
public void test52() throws Throwable {
    int int0 = NewSubject.extractIntInStr("");
    assertEquals(0, int0);
}

@Test(timeout = 4000)
public void test53() throws Throwable {
    int int0 = NewSubject.extractIntInStr((String) null);
    assertEquals(0, int0);
}

@Test(timeout = 4000)
public void test54() throws Throwable {
    char[] charArray0 = new char[5];
    charArray0[1] = 'A';
    String string0 = NewSubject.parseToken("Aug", charArray0);
}

```

```

        assertEquals("", string0);
        assertEquals(5, charArray0.length);
        assertNotNull(string0);
        assertEquals(new char[] {'\u0000', 'A', '\u0000', '\u0000', '\u0000'},
charArray0);
    }

    @Test(timeout = 4000)
    public void test55() throws Throwable {
        char[] charArray0 = new char[6];
        String string0 = NewSubject.parseToken("(invalid)", charArray0);
        assertEquals("(invalid)", string0);
        assertEquals(6, charArray0.length);
        assertNotNull(string0);
        assertEquals(new char[] {'\u0000', '\u0000', '\u0000', '\u0000', '\u0000',
'\u0000'}, charArray0);
    }

    @Test(timeout = 4000)
    public void test56() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("(invalid)", "Oct");
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test57() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("N", "Jun");
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test58() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("'Y~", (String) null);
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test59() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("$S3P&,*V+D&1{%U6",
"$S3P&,*V+D&1{%U6");
        assertTrue(boolean0);
    }

    @Test(timeout = 4000)
    public void test60() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase((String) null, "Apr");
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test61() throws Throwable {
        NewSubject newSubject0 = new NewSubject();
        assertNotNull(newSubject0);
    }
}

```

Raw LLM response (5110 chars)

```

```java
package comp5111.assignment.cut;

```

```

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf). The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's
 * generated tests pin down the *intended* behaviour, against which the
 * LLM's regenerated code is judged.
 *
 * Every signature is identical to the corresponding method in Subject.
 */
public class NewSubject {

 // ---- 1 ----
 public static boolean startsWithIgnoreCase(String str, String prefix) {
 if (str == null || prefix == null) {
 return false;
 }
 return str.toLowerCase().startsWith(prefix.toLowerCase());
 }

 // ---- 2 ----
 public static String parseToken(final String str, final char[] terminators) {
 if (str == null || terminators == null) {
 return str;
 }
 for (int i = 0; i < str.length(); i++) {
 for (char terminator : terminators) {
 if (str.charAt(i) == terminator) {
 return str.substring(0, i);
 }
 }
 }
 return str;
 }

 private static boolean isOneOf(char ch, final char[] chararray) {
 for (char c : chararray) {
 if (c == ch) {
 return true;
 }
 }
 return false;
 }

 // ---- 3 ----
 public static int extractIntInStr(String str) {
 if (str == null || str.isEmpty()) {
 return 0;
 }
 StringBuilder digits = new StringBuilder();
 for (int i = str.length() - 1; i >= 0; i--) {
 char ch = str.charAt(i);
 if (Character.isDigit(ch)) {
 digits.append(ch);
 } else if (digits.length() > 0) {
 break;
 }
 }
 return digits.length() > 0 ? Integer.parseInt(digits.reverse().toString()) : 0;
 }

 // ---- 4 ----
 public static int[] getVersionNo(final String versionString) {

```

```

 if (versionString == null || versionString.isEmpty()) {
 return null;
 }
 String[] parts = versionString.split("\\.");
 if (parts.length > 4) {
 return null;
 }
 int[] versionNumbers = new int[4];
 for (int i = 0; i < parts.length; i++) {
 try {
 versionNumbers[i] = Integer.parseInt(parts[i]);
 } catch (NumberFormatException e) {
 return null;
 }
 }
 return versionNumbers;
 }

 // ---- 5 ----
 public static String padLeft(String str, short length, char padChar) {
 if (str == null) {
 str = "";
 }
 if (str.length() >= length) {
 return str;
 }
 StringBuilder padded = new StringBuilder();
 for (int i = str.length(); i < length; i++) {
 padded.append(padChar);
 }
 padded.append(str);
 return padded.toString();
 }

 // ---- 6 ----
 public static boolean judgeLeapYear(int year) {
 if (year < 0) {
 return false;
 }
 return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
 }

 // ---- 7 ----
 public static int calcDaysInMonth(int year, int month) {
 if (month < 1 || month > 12) {
 return -1;
 }
 switch (month) {
 case 1: case 3: case 5: case 7: case 8: case 10: case 12:
 return 31;
 case 4: case 6: case 9: case 11:
 return 30;
 case 2:
 return judgeLeapYear(year) ? 29 : 28;
 default:
 return -1;
 }
 }

 // ---- 8 ----
 public static int getQuarter(int month) {
 if (month < 1 || month > 12) {
 return -1;
 }
 }

```

```

 return (month - 1) / 3 + 1;
 }

 // ---- 9 ----
 public static int monAbbr2month(String abbr) {
 if (abbr == null || abbr.length() != 3) {
 return -1;
 }
 switch (abbr) {
 case "Jan": return 1;
 case "Feb": return 2;
 case "Mar": return 3;
 case "Apr": return 4;
 case "May": return 5;
 case "Jun": return 6;
 case "Jul": return 7;
 case "Aug": return 8;
 case "Sep": return 9;
 case "Oct": return 10;
 case "Nov": return 11;
 case "Dec": return 12;
 default: return -1;
 }
 }

 // ---- 10 ----
 public static String month2MonAbbr(int month) {
 switch (month) {
 case 1: return "Jan";
 case 2: return "Feb";
 case 3: return "Mar";
 case 4: return "Apr";
 case 5: return "May";
 case 6: return "Jun";
 case 7: return "Jul";
 case 8: return "Aug";
 case 9: return "Sep";
 case 10: return "Oct";
 case 11: return "Nov";
 case 12: return "Dec";
 default: return "(invalid)";
 }
 }
}

```

### Extracted NewSubject.java for this round

```

```java
package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf). The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's
 * generated tests pin down the intended behaviour, against which the
 * LLM's regenerated code is judged.
 *
 * Every signature is identical to the corresponding method in Subject.
 */
public class NewSubject {

```

```

// ---- 1 ----
public static boolean startsWithIgnoreCase(String str, String prefix) {
    if (str == null || prefix == null) {
        return false;
    }
    return str.toLowerCase().startsWith(prefix.toLowerCase());
}

// ---- 2 ----
public static String parseToken(final String str, final char[] terminators) {
    if (str == null || terminators == null) {
        return str;
    }
    for (int i = 0; i < str.length(); i++) {
        for (char terminator : terminators) {
            if (str.charAt(i) == terminator) {
                return str.substring(0, i);
            }
        }
    }
    return str;
}

private static boolean isOneOf(char ch, final char[] chararray) {
    for (char c : chararray) {
        if (c == ch) {
            return true;
        }
    }
    return false;
}

// ---- 3 ----
public static int extractIntInStr(String str) {
    if (str == null || str.isEmpty()) {
        return 0;
    }
    StringBuilder digits = new StringBuilder();
    for (int i = str.length() - 1; i >= 0; i--) {
        char ch = str.charAt(i);
        if (Character.isDigit(ch)) {
            digits.append(ch);
        } else if (digits.length() > 0) {
            break;
        }
    }
    return digits.length() > 0 ? Integer.parseInt(digits.reverse().toString()) : 0;
}

// ---- 4 ----
public static int[] getVersionNo(final String versionString) {
    if (versionString == null || versionString.isEmpty()) {
        return null;
    }
    String[] parts = versionString.split("\\.");
    if (parts.length > 4) {
        return null;
    }
    int[] versionNumbers = new int[4];
    for (int i = 0; i < parts.length; i++) {
        try {
            versionNumbers[i] = Integer.parseInt(parts[i]);
        } catch (NumberFormatException e) {

```

```

        return null;
    }
}
return versionNumbers;
}

// ---- 5 ----
public static String padLeft(String str, short length, char padChar) {
    if (str == null) {
        str = "";
    }
    if (str.length() >= length) {
        return str;
    }
    StringBuilder padded = new StringBuilder();
    for (int i = str.length(); i < length; i++) {
        padded.append(padChar);
    }
    padded.append(str);
    return padded.toString();
}

// ---- 6 ----
public static boolean judgeLeapYear(int year) {
    if (year < 0) {
        return false;
    }
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

// ---- 7 ----
public static int calcDaysInMonth(int year, int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    switch (month) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            return 31;
        case 4: case 6: case 9: case 11:
            return 30;
        case 2:
            return judgeLeapYear(year) ? 29 : 28;
        default:
            return -1;
    }
}

// ---- 8 ----
public static int getQuarter(int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    return (month - 1) / 3 + 1;
}

// ---- 9 ----
public static int monAbbr2month(String abbr) {
    if (abbr == null || abbr.length() != 3) {
        return -1;
    }
    switch (abbr) {
        case "Jan": return 1;
        case "Feb": return 2;
        case "Mar": return 3;
    }
}

```

```

        case "Apr": return 4;
        case "May": return 5;
        case "Jun": return 6;
        case "Jul": return 7;
        case "Aug": return 8;
        case "Sep": return 9;
        case "Oct": return 10;
        case "Nov": return 11;
        case "Dec": return 12;
        default: return -1;
    }
}

// ---- 10 ----
public static String month2MonAbbr(int month) {
    switch (month) {
        case 1: return "Jan";
        case 2: return "Feb";
        case 3: return "Mar";
        case 4: return "Apr";
        case 5: return "May";
        case 6: return "Jun";
        case 7: return "Jul";
        case 8: return "Aug";
        case 9: return "Sep";
        case 10: return "Oct";
        case 11: return "Nov";
        case 12: return "Dec";
        default: return "(invalid)";
    }
}
}

```

Round 2

Test outcome

```

{
  "compile_ok": true,
  "run_count": 62,
  "fail_count": 2,
  "failures": [
    "test51(comp5111.assignment.cut.NewSubject_ESTest) :: null",
    "test43(comp5111.assignment.cut.NewSubject_ESTest) :: expected:<29> but was:<28>"
  ],
  "error": "",
  "raw_stdout": "10:33:28.315 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject_ESTest\n10:33:28.317 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject_ESTest'.\n10:33:28.344 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject_ESTest_scaffolding\n10:33:28.344 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject_ESTest_scaffolding'.\n10:33:28.346 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject_ESTest_scaffolding\n10:33:28.346 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject_ESTest\n10:33:28.517 [main] DEBUG

```



```

org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread: \"Reference
Handler\"\\n10:33:28.517 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Finalizer\"\\n10:33:28.517 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread: \"Signal
Dispatcher\"\\n10:33:28.517 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Java2D Disposer\"\\n10:33:28.517 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread:
\\n10:33:28.518 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Common-Cleaner\"\\n10:33:28.535 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject\\n10:33:28.535 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject'.\\n10:33:28.540 [main] INFO
org.evosuite.runtime.instrumentation.MethodCallReplacementClassAdapter - Adding mock
interface to class comp5111/assignment/cut/NewSubject\\n10:33:28.610 [main] INFO
org.evosuite.runtime.instrumentation.CreateClassResetClassAdapter - Creating brand-new
static initializer in class comp5111/assignment/cut/NewSubject\\n10:33:28.611 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject\\n10:33:28.628 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:28.647 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:28.649 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:28.650 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:28.651 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:28.652 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:28.653 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:28.656 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new
thread\\nRUN_COUNT=62\\nFAIL_COUNT=2\\nFAILURE>>>test51(comp5111.assignment.cut.NewSubject_EST
null<<<FAILURE\\nFAILURE>>>test43(comp5111.assignment.cut.NewSubject_ESTest) ::
expected:<29> but was:<28><<<FAILURE\\n\",
  \"raw_stderr\": \"WARNING: An illegal reflective access operation has occurred\\nWARNING:
Illegal reflective access by org.evosuite.runtime.GuiSupport (file:/home/zimo/playground/
COMP5111-2026Spring-Assignments-Students/lib/evosuite-1.2.0.jar) to field
java.awt.GraphicsEnvironment.headless\\nWARNING: Please consider reporting this to the
maintainers of org.evosuite.runtime.GuiSupport\\nWARNING: Use --illegal-access=warn to
enable warnings of further illegal reflective access operations\\nWARNING: All illegal
access operations will be denied in a future release\\n\"
}

```

Prompt sent to LLM (20628 chars)

You are a Java engineer. Implement the class `comp5111.assignment.cut.NewSubject` so that every JUnit test in the provided test suite passes.

Rules:

- Output exactly ONE fenced ``java code block containing the FULL contents of NewSubject.java (package, imports if any, the entire class).
- Do NOT modify the test suite. Do NOT add main(). Keep the package and class name unchanged. Method signatures shown below are mandatory.
- Output nothing outside the single code block.

=== Method signatures + Javadoc (mandatory shape) ===

```
package comp5111.assignment.cut;
```

```
/**
```

```
 * Task 4 reduced CUT.
```

```
 *
```

```

 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf). The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's

```

```

* generated tests pin down the *intended* behaviour, against which the
* LLM's regenerated code is judged.
*
* Every signature is identical to the corresponding method in Subject.
*/
public class NewSubject {

    // ---- 1 ----
    /**
     * Returns true if str starts with prefix ignoring case, else false.
     * Returns false on any null argument or if str is shorter than prefix.
     */
    public static boolean startsWithIgnoreCase(String str, String prefix);

    // ---- 2 ----
    /**
     * Returns the prefix of str up to (but not including) the first
     * character that appears in {@code terminators}. If no terminator
     * is found, the whole string is returned.
     */
    public static String parseToken(final String str, final char[] terminators);

    private static boolean isOneOf(char ch, final char[] chararray);

    // ---- 3 ----
    /**
     * Returns the LAST contiguous run of digits in str interpreted as
     * an int. Empty/null/no-digit inputs return 0.
     * Examples: "1234" -> 1234, "a" -> 0, "1234a123" -> 123.
     */
    public static int extractIntInStr(String str);

    // ---- 4 ----
    /**
     * Parses a version string of form a, a.b, a.b.c, or a.b.c.d into a 4-int
     * array. Trailing components default to 0. More than 4 dot-segments or
     * malformed (consecutive dots) produces null.
     */
    public static int[] getVersionNo(final String versionString);

    // ---- 5 ----
    /**
     * Pads str on the left with padChar so the resulting length is at least
     * length. Null str is treated as "". If str is already long enough it is
     * returned unchanged.
     */
    public static String padLeft(String str, short length, char padChar);

    // ---- 6 ----
    /** Returns true iff year is a leap year (Gregorian rule). */
    public static boolean judgeLeapYear(int year);

    // ---- 7 ----
    /**
     * Returns the number of days in the given month (1-12) of the given
     * year. Returns -1 if month is out of range. February uses leap-year
     * rules.
     */
    public static int calcDaysInMonth(int year, int month);

    // ---- 8 ----
    /** Returns the calendar quarter (1-4) for month (1-12), or -1 if invalid. */
    public static int getQuarter(int month);

```

```

// ---- 9 ----
/**
 * Converts a 3-char month abbreviation (case-sensitive, e.g. "Jan",
 * "Sep") into its month number (1-12). Returns -1 on null, wrong
 * length, or unknown abbreviation.
 */
public static int monAbbr2month(String abbr);

// ---- 10 ----
/**
 * Inverse of monAbbr2month: returns "Jan".. "Dec" for month 1..12, and
 * "(invalid)" otherwise.
 */
public static String month2MonAbbr(int month);
}

```

=== EvoSuite-generated test suite (passes against the ground truth) ===

```

/*
 * This file was automatically generated by EvoSuite
 * Thu May 07 11:32:05 GMT 2026
 */

package comp5111.assignment.cut;

import org.junit.Test;
import static org.junit.Assert.*;
import comp5111.assignment.cut.NewSubject;
import org.evosuite.runtime.EvoRunner;
import org.evosuite.runtime.EvoRunnerParameters;
import org.junit.runner.RunWith;

@RunWith(EvoRunner.class) @EvoRunnerParameters(mockJVMNonDeterminism = true, useVFS =
true, useVNET = true, resetStaticState = true, separateClassLoader = true)
public class NewSubject_ESTest extends NewSubject_ESTest_scaffolding {

    @Test(timeout = 4000)
    public void test00() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(1493);
        assertEquals("(invalid)", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test01() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(12);
        assertEquals("Dec", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test02() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(11);
        assertEquals("Nov", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test03() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(10);
        assertEquals("Oct", string0);
        assertNotNull(string0);
    }
}

```

```
@Test(timeout = 4000)
public void test04() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(9);
    assertEquals("Sep", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test05() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(8);
    assertEquals("Aug", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test06() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(7);
    assertEquals("Jul", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test07() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(6);
    assertEquals("Jun", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test08() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(5);
    assertEquals("May", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test09() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(4);
    assertEquals("Apr", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test10() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(3);
    assertEquals("Mar", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test11() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(2);
    assertEquals("Feb", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test12() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(1);
    assertEquals("Jan", string0);
    assertNotNull(string0);
}
```

```
@Test(timeout = 4000)
public void test13() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Dec");
    assertEquals(12, int0);
}

@Test(timeout = 4000)
public void test14() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Nov");
    assertEquals(11, int0);
}

@Test(timeout = 4000)
public void test15() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Oct");
    assertEquals(10, int0);
}

@Test(timeout = 4000)
public void test16() throws Throwable {
    int int0 = NewSubject.monAbbr2month("'Y~");
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test17() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Aug");
    assertEquals(8, int0);
}

@Test(timeout = 4000)
public void test18() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Jul");
    assertEquals(7, int0);
}

@Test(timeout = 4000)
public void test19() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Jun");
    assertEquals(6, int0);
}

@Test(timeout = 4000)
public void test20() throws Throwable {
    int int0 = NewSubject.monAbbr2month("May");
    assertEquals(5, int0);
}

@Test(timeout = 4000)
public void test21() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Apr");
    assertEquals(4, int0);
}

@Test(timeout = 4000)
public void test22() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Mar");
    assertEquals(3, int0);
}

@Test(timeout = 4000)
public void test23() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Sep");
    assertEquals(9, int0);
}
```

```

}

@Test(timeout = 4000)
public void test24() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Jan");
    assertEquals(1, int0);
}

@Test(timeout = 4000)
public void test25() throws Throwable {
    int int0 = NewSubject.monAbbr2month("(invalid)");
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test26() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Feb");
    assertEquals(2, int0);
}

@Test(timeout = 4000)
public void test27() throws Throwable {
    int int0 = NewSubject.monAbbr2month((String) null);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test28() throws Throwable {
    int int0 = NewSubject.getQuarter(11);
    assertEquals(4, int0);
}

@Test(timeout = 4000)
public void test29() throws Throwable {
    int int0 = NewSubject.getQuarter(4);
    assertEquals(2, int0);
}

@Test(timeout = 4000)
public void test30() throws Throwable {
    int int0 = NewSubject.getQuarter((short)8);
    assertEquals(3, int0);
}

@Test(timeout = 4000)
public void test31() throws Throwable {
    int int0 = NewSubject.getQuarter(946);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test32() throws Throwable {
    int int0 = NewSubject.getQuarter(2);
    assertEquals(1, int0);
}

@Test(timeout = 4000)
public void test33() throws Throwable {
    int int0 = NewSubject.getQuarter((-1460));
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test34() throws Throwable {

```

```

        int int0 = NewSubject.calcDaysInMonth(11, 11);
        assertEquals(30, int0);
    }

    @Test(timeout = 4000)
    public void test35() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth((-1), 9);
        assertEquals(30, int0);
    }

    @Test(timeout = 4000)
    public void test36() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth(0, 6);
        assertEquals(30, int0);
    }

    @Test(timeout = 4000)
    public void test37() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth(2, 4);
        assertEquals(30, int0);
    }

    @Test(timeout = 4000)
    public void test38() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth((-1), 2);
        assertEquals(28, int0);
    }

    @Test(timeout = 4000)
    public void test39() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth(0, 3);
        assertEquals(31, int0);
    }

    @Test(timeout = 4000)
    public void test40() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth(1, 1726);
        assertEquals((-1), int0);
    }

    @Test(timeout = 4000)
    public void test41() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth((-3324), (-1570));
        assertEquals((-1), int0);
    }

    @Test(timeout = 4000)
    public void test42() throws Throwable {
        boolean boolean0 = NewSubject.judgeLeapYear(3300);
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test43() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth((-3324), 2);
        assertEquals(29, int0);
    }

    @Test(timeout = 4000)
    public void test44() throws Throwable {
        boolean boolean0 = NewSubject.judgeLeapYear(0);
        assertTrue(boolean0);
    }

```

```

@Test(timeout = 4000)
public void test45() throws Throwable {
    String string0 = NewSubject.padLeft("Jun", (short)1741, 'G');
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test46() throws Throwable {
    String string0 = NewSubject.padLeft((String) null, (short) (-1), '(');
    assertEquals("", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test47() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo(".:r~w'iW");
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test48() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo("oZ.>B6oZ.>B6oZ.>B6oZ.>B6");
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test49() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo("");
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test50() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo((String) null);
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test51() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo("$S3P&,*V+D&1{%U6");
    assertEquals(4, intArray0.length);
    assertNotNull(intArray0);
    assertEquals(new int[] {6, 0, 0, 0}, intArray0);
}

@Test(timeout = 4000)
public void test52() throws Throwable {
    int int0 = NewSubject.extractIntInStr("");
    assertEquals(0, int0);
}

@Test(timeout = 4000)
public void test53() throws Throwable {
    int int0 = NewSubject.extractIntInStr((String) null);
    assertEquals(0, int0);
}

@Test(timeout = 4000)
public void test54() throws Throwable {
    char[] charArray0 = new char[5];
    charArray0[1] = 'A';
    String string0 = NewSubject.parseToken("Aug", charArray0);
    assertEquals("", string0);
    assertEquals(5, charArray0.length);
}

```



```

        assertNotNull(string0);
        assertEquals(new char[] {'\u0000', 'A', '\u0000', '\u0000', '\u0000'},
charArray0);
    }

    @Test(timeout = 4000)
    public void test55() throws Throwable {
        char[] charArray0 = new char[6];
        String string0 = NewSubject.parseToken("(invalid)", charArray0);
        assertEquals("(invalid)", string0);
        assertEquals(6, charArray0.length);
        assertNotNull(string0);
        assertEquals(new char[] {'\u0000', '\u0000', '\u0000', '\u0000', '\u0000',
'\u0000'}, charArray0);
    }

    @Test(timeout = 4000)
    public void test56() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("(invalid)", "Oct");
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test57() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("N", "Jun");
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test58() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("'Y~", (String) null);
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test59() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("$S3P&,*V+D&1{%U6",
"$S3P&,*V+D&1{%U6");
        assertTrue(boolean0);
    }

    @Test(timeout = 4000)
    public void test60() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase((String) null, "Apr");
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test61() throws Throwable {
        NewSubject newSubject0 = new NewSubject();
        assertNotNull(newSubject0);
    }
}

```

=== Your previous implementation (round 1) ===

```
package comp5111.assignment.cut;
```

```

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf). The patched

```

```

* (correct) implementations from Task 2 are used so that EvoSuite's
* generated tests pin down the *intended* behaviour, against which the
* LLM's regenerated code is judged.
*
* Every signature is identical to the corresponding method in Subject.
*/
public class NewSubject {

    // ---- 1 ----
    public static boolean startsWithIgnoreCase(String str, String prefix) {
        if (str == null || prefix == null) {
            return false;
        }
        return str.toLowerCase().startsWith(prefix.toLowerCase());
    }

    // ---- 2 ----
    public static String parseToken(final String str, final char[] terminators) {
        if (str == null || terminators == null) {
            return str;
        }
        for (int i = 0; i < str.length(); i++) {
            for (char terminator : terminators) {
                if (str.charAt(i) == terminator) {
                    return str.substring(0, i);
                }
            }
        }
        return str;
    }

    private static boolean isOneOf(char ch, final char[] chararray) {
        for (char c : chararray) {
            if (c == ch) {
                return true;
            }
        }
        return false;
    }

    // ---- 3 ----
    public static int extractIntInStr(String str) {
        if (str == null || str.isEmpty()) {
            return 0;
        }
        StringBuilder digits = new StringBuilder();
        for (int i = str.length() - 1; i >= 0; i--) {
            char ch = str.charAt(i);
            if (Character.isDigit(ch)) {
                digits.append(ch);
            } else if (digits.length() > 0) {
                break;
            }
        }
        return digits.length() > 0 ? Integer.parseInt(digits.reverse().toString()) : 0;
    }

    // ---- 4 ----
    public static int[] getVersionNo(final String versionString) {
        if (versionString == null || versionString.isEmpty()) {
            return null;
        }
        String[] parts = versionString.split("\\.");
        if (parts.length > 4) {

```

```

        return null;
    }
    int[] versionNumbers = new int[4];
    for (int i = 0; i < parts.length; i++) {
        try {
            versionNumbers[i] = Integer.parseInt(parts[i]);
        } catch (NumberFormatException e) {
            return null;
        }
    }
    return versionNumbers;
}

// ---- 5 ----
public static String padLeft(String str, short length, char padChar) {
    if (str == null) {
        str = "";
    }
    if (str.length() >= length) {
        return str;
    }
    StringBuilder padded = new StringBuilder();
    for (int i = str.length(); i < length; i++) {
        padded.append(padChar);
    }
    padded.append(str);
    return padded.toString();
}

// ---- 6 ----
public static boolean judgeLeapYear(int year) {
    if (year < 0) {
        return false;
    }
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

// ---- 7 ----
public static int calcDaysInMonth(int year, int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    switch (month) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            return 31;
        case 4: case 6: case 9: case 11:
            return 30;
        case 2:
            return judgeLeapYear(year) ? 29 : 28;
        default:
            return -1;
    }
}

// ---- 8 ----
public static int getQuarter(int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    return (month - 1) / 3 + 1;
}

// ---- 9 ----
public static int monAbbr2month(String abbr) {

```

```

        if (abbr == null || abbr.length() != 3) {
            return -1;
        }
        switch (abbr) {
            case "Jan": return 1;
            case "Feb": return 2;
            case "Mar": return 3;
            case "Apr": return 4;
            case "May": return 5;
            case "Jun": return 6;
            case "Jul": return 7;
            case "Aug": return 8;
            case "Sep": return 9;
            case "Oct": return 10;
            case "Nov": return 11;
            case "Dec": return 12;
            default: return -1;
        }
    }

    // ---- 10 ----
    public static String month2MonAbbr(int month) {
        switch (month) {
            case 1: return "Jan";
            case 2: return "Feb";
            case 3: return "Mar";
            case 4: return "Apr";
            case 5: return "May";
            case 6: return "Jun";
            case 7: return "Jul";
            case 8: return "Aug";
            case 9: return "Sep";
            case 10: return "Oct";
            case 11: return "Nov";
            case 12: return "Dec";
            default: return "(invalid)";
        }
    }
}

```

=== Test failures observed for that implementation ===

2 of 62 tests failed.

- test51(comp5111.assignment.cut.NewSubject_ESTest) :: null
- test43(comp5111.assignment.cut.NewSubject_ESTest) :: expected:<29> but was:<28>

Produce a corrected NewSubject.java that fixes ALL of the failures above without breaking previously passing tests.

Raw LLM response (5110 chars)

```

```java
package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf). The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's

```

```

* generated tests pin down the *intended* behaviour, against which the
* LLM's regenerated code is judged.
*
* Every signature is identical to the corresponding method in Subject.
*/
public class NewSubject {

 // ---- 1 ----
 public static boolean startsWithIgnoreCase(String str, String prefix) {
 if (str == null || prefix == null) {
 return false;
 }
 return str.toLowerCase().startsWith(prefix.toLowerCase());
 }

 // ---- 2 ----
 public static String parseToken(final String str, final char[] terminators) {
 if (str == null || terminators == null) {
 return str;
 }
 for (int i = 0; i < str.length(); i++) {
 for (char terminator : terminators) {
 if (str.charAt(i) == terminator) {
 return str.substring(0, i);
 }
 }
 }
 return str;
 }

 private static boolean isOneOf(char ch, final char[] chararray) {
 for (char c : chararray) {
 if (c == ch) {
 return true;
 }
 }
 return false;
 }

 // ---- 3 ----
 public static int extractIntInStr(String str) {
 if (str == null || str.isEmpty()) {
 return 0;
 }
 StringBuilder digits = new StringBuilder();
 for (int i = str.length() - 1; i >= 0; i--) {
 char ch = str.charAt(i);
 if (Character.isDigit(ch)) {
 digits.append(ch);
 } else if (digits.length() > 0) {
 break;
 }
 }
 return digits.length() > 0 ? Integer.parseInt(digits.reverse().toString()) : 0;
 }

 // ---- 4 ----
 public static int[] getVersionNo(final String versionString) {
 if (versionString == null || versionString.isEmpty()) {
 return null;
 }
 String[] parts = versionString.split("\\.");
 if (parts.length > 4) {
 return null;
 }
 }
}

```

```

 }
 int[] versionNumbers = new int[4];
 for (int i = 0; i < parts.length; i++) {
 try {
 versionNumbers[i] = Integer.parseInt(parts[i]);
 } catch (NumberFormatException e) {
 return null;
 }
 }
 return versionNumbers;
}

// ---- 5 ----
public static String padLeft(String str, short length, char padChar) {
 if (str == null) {
 str = "";
 }
 if (str.length() >= length) {
 return str;
 }
 StringBuilder padded = new StringBuilder();
 for (int i = str.length(); i < length; i++) {
 padded.append(padChar);
 }
 padded.append(str);
 return padded.toString();
}

// ---- 6 ----
public static boolean judgeLeapYear(int year) {
 if (year < 0) {
 return false;
 }
 return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

// ---- 7 ----
public static int calcDaysInMonth(int year, int month) {
 if (month < 1 || month > 12) {
 return -1;
 }
 switch (month) {
 case 1: case 3: case 5: case 7: case 8: case 10: case 12:
 return 31;
 case 4: case 6: case 9: case 11:
 return 30;
 case 2:
 return judgeLeapYear(year) ? 29 : 28;
 default:
 return -1;
 }
}

// ---- 8 ----
public static int getQuarter(int month) {
 if (month < 1 || month > 12) {
 return -1;
 }
 return (month - 1) / 3 + 1;
}

// ---- 9 ----
public static int monAbbr2month(String abbr) {
 if (abbr == null || abbr.length() != 3) {

```

```

 return -1;
 }
 switch (abbr) {
 case "Jan": return 1;
 case "Feb": return 2;
 case "Mar": return 3;
 case "Apr": return 4;
 case "May": return 5;
 case "Jun": return 6;
 case "Jul": return 7;
 case "Aug": return 8;
 case "Sep": return 9;
 case "Oct": return 10;
 case "Nov": return 11;
 case "Dec": return 12;
 default: return -1;
 }
}

// ---- 10 ----
public static String month2MonAbbr(int month) {
 switch (month) {
 case 1: return "Jan";
 case 2: return "Feb";
 case 3: return "Mar";
 case 4: return "Apr";
 case 5: return "May";
 case 6: return "Jun";
 case 7: return "Jul";
 case 8: return "Aug";
 case 9: return "Sep";
 case 10: return "Oct";
 case 11: return "Nov";
 case 12: return "Dec";
 default: return "(invalid)";
 }
}
}

```

### Extracted NewSubject.java for this round

```

```java
package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf). The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's
 * generated tests pin down the intended behaviour, against which the
 * LLM's regenerated code is judged.
 *
 * Every signature is identical to the corresponding method in Subject.
 */
public class NewSubject {

    // ---- 1 ----
    public static boolean startsWithIgnoreCase(String str, String prefix) {
        if (str == null || prefix == null) {
            return false;
        }
    }
}

```

```

        return str.toLowerCase().startsWith(prefix.toLowerCase());
    }

    // ---- 2 ----
    public static String parseToken(final String str, final char[] terminators) {
        if (str == null || terminators == null) {
            return str;
        }
        for (int i = 0; i < str.length(); i++) {
            for (char terminator : terminators) {
                if (str.charAt(i) == terminator) {
                    return str.substring(0, i);
                }
            }
        }
        return str;
    }

    private static boolean isOneOf(char ch, final char[] chararray) {
        for (char c : chararray) {
            if (c == ch) {
                return true;
            }
        }
        return false;
    }

    // ---- 3 ----
    public static int extractIntInStr(String str) {
        if (str == null || str.isEmpty()) {
            return 0;
        }
        StringBuilder digits = new StringBuilder();
        for (int i = str.length() - 1; i >= 0; i--) {
            char ch = str.charAt(i);
            if (Character.isDigit(ch)) {
                digits.append(ch);
            } else if (digits.length() > 0) {
                break;
            }
        }
        return digits.length() > 0 ? Integer.parseInt(digits.reverse().toString()) : 0;
    }

    // ---- 4 ----
    public static int[] getVersionNo(final String versionString) {
        if (versionString == null || versionString.isEmpty()) {
            return null;
        }
        String[] parts = versionString.split("\\.");
        if (parts.length > 4) {
            return null;
        }
        int[] versionNumbers = new int[4];
        for (int i = 0; i < parts.length; i++) {
            try {
                versionNumbers[i] = Integer.parseInt(parts[i]);
            } catch (NumberFormatException e) {
                return null;
            }
        }
        return versionNumbers;
    }
}

```



```

// ---- 5 ----
public static String padLeft(String str, short length, char padChar) {
    if (str == null) {
        str = "";
    }
    if (str.length() >= length) {
        return str;
    }
    StringBuilder padded = new StringBuilder();
    for (int i = str.length(); i < length; i++) {
        padded.append(padChar);
    }
    padded.append(str);
    return padded.toString();
}

// ---- 6 ----
public static boolean judgeLeapYear(int year) {
    if (year < 0) {
        return false;
    }
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

// ---- 7 ----
public static int calcDaysInMonth(int year, int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    switch (month) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            return 31;
        case 4: case 6: case 9: case 11:
            return 30;
        case 2:
            return judgeLeapYear(year) ? 29 : 28;
        default:
            return -1;
    }
}

// ---- 8 ----
public static int getQuarter(int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    return (month - 1) / 3 + 1;
}

// ---- 9 ----
public static int monAbbr2month(String abbr) {
    if (abbr == null || abbr.length() != 3) {
        return -1;
    }
    switch (abbr) {
        case "Jan": return 1;
        case "Feb": return 2;
        case "Mar": return 3;
        case "Apr": return 4;
        case "May": return 5;
        case "Jun": return 6;
        case "Jul": return 7;
        case "Aug": return 8;
        case "Sep": return 9;
    }
}

```

```

        case "Oct": return 10;
        case "Nov": return 11;
        case "Dec": return 12;
        default: return -1;
    }
}

// ---- 10 ----
public static String month2MonAbbr(int month) {
    switch (month) {
        case 1: return "Jan";
        case 2: return "Feb";
        case 3: return "Mar";
        case 4: return "Apr";
        case 5: return "May";
        case 6: return "Jun";
        case 7: return "Jul";
        case 8: return "Aug";
        case 9: return "Sep";
        case 10: return "Oct";
        case 11: return "Nov";
        case 12: return "Dec";
        default: return "(invalid)";
    }
}
}

```

Round 3

Test outcome

```

{
  "compile_ok": true,
  "run_count": 62,
  "fail_count": 2,
  "failures": [
    "test51(comp5111.assignment.cut.NewSubject_ESTest) :: null",
    "test43(comp5111.assignment.cut.NewSubject_ESTest) :: expected:<29> but was:<28>"
  ],
  "error": "",
  "raw stdout": "s for first time:
comp5111.assignment.cut.NewSubject_ESTest\n10:33:43.502 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject_ESTest'.\n10:33:43.529 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject_ESTest_scaffolding\n10:33:43.529 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject_ESTest_scaffolding'.\n10:33:43.531 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject_ESTest_scaffolding\n10:33:43.531 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject_ESTest\n10:33:43.708 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread: \"Reference
Handler\"\n10:33:43.709 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Finalizer\"\n10:33:43.709 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread: \"Signal
Dispatcher\"\n10:33:43.709 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Java2D Disposer\"\n10:33:43.709 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread:

```

```

\"main\"\\n10:33:43.709 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Common-Cleaner\"\\n10:33:43.727 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject\\n10:33:43.727 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject'.\\n10:33:43.733 [main] INFO
org.evosuite.runtime.instrumentation.MethodCallReplacementClassAdapter - Adding mock
interface to class comp5111/assignment/cut/NewSubject\\n10:33:43.804 [main] INFO
org.evosuite.runtime.instrumentation.CreateClassResetClassAdapter - Creating brand-new
static initializer in class comp5111/assignment/cut/NewSubject\\n10:33:43.805 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject\\n10:33:43.817 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.821 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.822 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.833 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.840 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.841 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.842 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.843 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.846 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.847 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.848 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.848 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.849 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.850 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.851 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.852 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.852 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:43.853 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new
thread\\nRUN_COUNT=62\\nFAIL_COUNT=2\\nFAILURE>>>test51(comp5111.assignment.cut.NewSubject_EST
null<<<FAILURE\\nFAILURE>>>test43(comp5111.assignment.cut.NewSubject_ESTest) ::
expected:<29> but was:<28><<<FAILURE\\n\",
  \"raw_stderr\": \"WARNING: An illegal reflective access operation has occurred\\nWARNING:
Illegal reflective access by org.evosuite.runtime.GuiSupport (file:/home/zimo/playground/
COMP5111-2026Spring-Assignments-Students/lib/evosuite-1.2.0.jar) to field
java.awt.GraphicsEnvironment.headless\\nWARNING: Please consider reporting this to the
maintainers of org.evosuite.runtime.GuiSupport\\nWARNING: Use --illegal-access=warn to
enable warnings of further illegal reflective access operations\\nWARNING: All illegal
access operations will be denied in a future release\\n\"
}

```

Prompt sent to LLM (20628 chars)

You are a Java engineer. Implement the class `comp5111.assignment.cut.NewSubject` so that every JUnit test in the provided test suite passes.

Rules:

- Output exactly ONE fenced ``java code block containing the FULL contents of NewSubject.java (package, imports if any, the entire class).
- Do NOT modify the test suite. Do NOT add main(). Keep the package and class name unchanged. Method signatures shown below are mandatory.
- Output nothing outside the single code block.

=== Method signatures + Javadoc (mandatory shape) ===

```
package comp5111.assignment.cut;
```

```
/**
```

```
 * Task 4 reduced CUT.
```

```
 *
```

```

* Ten public-static methods selected from {@link Subject}, with their
* documented contracts and one private helper (isOneOf). The patched
* (correct) implementations from Task 2 are used so that EvoSuite's
* generated tests pin down the *intended* behaviour, against which the
* LLM's regenerated code is judged.
*
* Every signature is identical to the corresponding method in Subject.
*/
public class NewSubject {

    // ---- 1 ----
    /**
     * Returns true if str starts with prefix ignoring case, else false.
     * Returns false on any null argument or if str is shorter than prefix.
     */
    public static boolean startsWithIgnoreCase(String str, String prefix);

    // ---- 2 ----
    /**
     * Returns the prefix of str up to (but not including) the first
     * character that appears in {@code terminators}. If no terminator
     * is found, the whole string is returned.
     */
    public static String parseToken(final String str, final char[] terminators);

    private static boolean isOneOf(char ch, final char[] chararray);

    // ---- 3 ----
    /**
     * Returns the LAST contiguous run of digits in str interpreted as
     * an int. Empty/null/no-digit inputs return 0.
     * Examples: "1234" -> 1234, "a" -> 0, "1234a123" -> 123.
     */
    public static int extractIntInStr(String str);

    // ---- 4 ----
    /**
     * Parses a version string of form a, a.b, a.b.c, or a.b.c.d into a 4-int
     * array. Trailing components default to 0. More than 4 dot-segments or
     * malformed (consecutive dots) produces null.
     */
    public static int[] getVersionNo(final String versionString);

    // ---- 5 ----
    /**
     * Pads str on the left with padChar so the resulting length is at least
     * length. Null str is treated as "". If str is already long enough it is
     * returned unchanged.
     */
    public static String padLeft(String str, short length, char padChar);

    // ---- 6 ----
    /** Returns true iff year is a leap year (Gregorian rule). */
    public static boolean judgeLeapYear(int year);

    // ---- 7 ----
    /**
     * Returns the number of days in the given month (1-12) of the given
     * year. Returns -1 if month is out of range. February uses leap-year
     * rules.
     */
    public static int calcDaysInMonth(int year, int month);

    // ---- 8 ----

```

```

    /** Returns the calendar quarter (1-4) for month (1-12), or -1 if invalid. */
    public static int getQuarter(int month);

    // ---- 9 ----
    /**
     * Converts a 3-char month abbreviation (case-sensitive, e.g. "Jan",
     * "Sep") into its month number (1-12). Returns -1 on null, wrong
     * length, or unknown abbreviation.
     */
    public static int monAbbr2month(String abbr);

    // ---- 10 ----
    /**
     * Inverse of monAbbr2month: returns "Jan".. "Dec" for month 1..12, and
     * "(invalid)" otherwise.
     */
    public static String month2MonAbbr(int month);
}

```

=== EvoSuite-generated test suite (passes against the ground truth) ===

```

/*
 * This file was automatically generated by EvoSuite
 * Thu May 07 11:32:05 GMT 2026
 */

package comp5111.assignment.cut;

import org.junit.Test;
import static org.junit.Assert.*;
import comp5111.assignment.cut.NewSubject;
import org.evosuite.runtime.EvoRunner;
import org.evosuite.runtime.EvoRunnerParameters;
import org.junit.runner.RunWith;

@RunWith(EvoRunner.class) @EvoRunnerParameters(mockJVMNonDeterminism = true, useVFS =
true, useVNET = true, resetStaticState = true, separateClassLoader = true)
public class NewSubject_ESTest extends NewSubject_ESTest_scaffolding {

    @Test(timeout = 4000)
    public void test00() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(1493);
        assertEquals("(invalid)", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test01() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(12);
        assertEquals("Dec", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test02() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(11);
        assertEquals("Nov", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test03() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(10);
    }
}

```

```

        assertEquals("Oct", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test04() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(9);
        assertEquals("Sep", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test05() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(8);
        assertEquals("Aug", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test06() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(7);
        assertEquals("Jul", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test07() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(6);
        assertEquals("Jun", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test08() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(5);
        assertEquals("May", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test09() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(4);
        assertEquals("Apr", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test10() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(3);
        assertEquals("Mar", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test11() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(2);
        assertEquals("Feb", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test12() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(1);
        assertEquals("Jan", string0);
    }

```

```

        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test13() throws Throwable {
        int int0 = NewSubject.monAbbr2month("Dec");
        assertEquals(12, int0);
    }

    @Test(timeout = 4000)
    public void test14() throws Throwable {
        int int0 = NewSubject.monAbbr2month("Nov");
        assertEquals(11, int0);
    }

    @Test(timeout = 4000)
    public void test15() throws Throwable {
        int int0 = NewSubject.monAbbr2month("Oct");
        assertEquals(10, int0);
    }

    @Test(timeout = 4000)
    public void test16() throws Throwable {
        int int0 = NewSubject.monAbbr2month("'Y~");
        assertEquals((-1), int0);
    }

    @Test(timeout = 4000)
    public void test17() throws Throwable {
        int int0 = NewSubject.monAbbr2month("Aug");
        assertEquals(8, int0);
    }

    @Test(timeout = 4000)
    public void test18() throws Throwable {
        int int0 = NewSubject.monAbbr2month("Jul");
        assertEquals(7, int0);
    }

    @Test(timeout = 4000)
    public void test19() throws Throwable {
        int int0 = NewSubject.monAbbr2month("Jun");
        assertEquals(6, int0);
    }

    @Test(timeout = 4000)
    public void test20() throws Throwable {
        int int0 = NewSubject.monAbbr2month("May");
        assertEquals(5, int0);
    }

    @Test(timeout = 4000)
    public void test21() throws Throwable {
        int int0 = NewSubject.monAbbr2month("Apr");
        assertEquals(4, int0);
    }

    @Test(timeout = 4000)
    public void test22() throws Throwable {
        int int0 = NewSubject.monAbbr2month("Mar");
        assertEquals(3, int0);
    }

    @Test(timeout = 4000)

```

```

public void test23() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Sep");
    assertEquals(9, int0);
}

@Test(timeout = 4000)
public void test24() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Jan");
    assertEquals(1, int0);
}

@Test(timeout = 4000)
public void test25() throws Throwable {
    int int0 = NewSubject.monAbbr2month("(invalid)");
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test26() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Feb");
    assertEquals(2, int0);
}

@Test(timeout = 4000)
public void test27() throws Throwable {
    int int0 = NewSubject.monAbbr2month((String) null);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test28() throws Throwable {
    int int0 = NewSubject.getQuarter(11);
    assertEquals(4, int0);
}

@Test(timeout = 4000)
public void test29() throws Throwable {
    int int0 = NewSubject.getQuarter(4);
    assertEquals(2, int0);
}

@Test(timeout = 4000)
public void test30() throws Throwable {
    int int0 = NewSubject.getQuarter((short)8);
    assertEquals(3, int0);
}

@Test(timeout = 4000)
public void test31() throws Throwable {
    int int0 = NewSubject.getQuarter(946);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test32() throws Throwable {
    int int0 = NewSubject.getQuarter(2);
    assertEquals(1, int0);
}

@Test(timeout = 4000)
public void test33() throws Throwable {
    int int0 = NewSubject.getQuarter((-1460));
    assertEquals((-1), int0);
}

```



```

@Test(timeout = 4000)
public void test34() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth(11, 11);
    assertEquals(30, int0);
}

@Test(timeout = 4000)
public void test35() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth((-1), 9);
    assertEquals(30, int0);
}

@Test(timeout = 4000)
public void test36() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth(0, 6);
    assertEquals(30, int0);
}

@Test(timeout = 4000)
public void test37() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth(2, 4);
    assertEquals(30, int0);
}

@Test(timeout = 4000)
public void test38() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth((-1), 2);
    assertEquals(28, int0);
}

@Test(timeout = 4000)
public void test39() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth(0, 3);
    assertEquals(31, int0);
}

@Test(timeout = 4000)
public void test40() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth(1, 1726);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test41() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth((-3324), (-1570));
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test42() throws Throwable {
    boolean boolean0 = NewSubject.judgeLeapYear(3300);
    assertFalse(boolean0);
}

@Test(timeout = 4000)
public void test43() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth((-3324), 2);
    assertEquals(29, int0);
}

@Test(timeout = 4000)
public void test44() throws Throwable {
    boolean boolean0 = NewSubject.judgeLeapYear(0);
}

```

```

        assertTrue(boolean0);
    }

    @Test(timeout = 4000)
    public void test45() throws Throwable {
        String string0 = NewSubject.padLeft("Jun", (short)1741, 'G');
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test46() throws Throwable {
        String string0 = NewSubject.padLeft((String) null, (short) (-1), '(');
        assertEquals("", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test47() throws Throwable {
        int[] intArray0 = NewSubject.getVersionNo(".@:r~w'iW");
        assertNull(intArray0);
    }

    @Test(timeout = 4000)
    public void test48() throws Throwable {
        int[] intArray0 = NewSubject.getVersionNo("oZ.>B6oZ.>B6oZ.>B6oZ.>B6");
        assertNull(intArray0);
    }

    @Test(timeout = 4000)
    public void test49() throws Throwable {
        int[] intArray0 = NewSubject.getVersionNo("");
        assertNull(intArray0);
    }

    @Test(timeout = 4000)
    public void test50() throws Throwable {
        int[] intArray0 = NewSubject.getVersionNo((String) null);
        assertNull(intArray0);
    }

    @Test(timeout = 4000)
    public void test51() throws Throwable {
        int[] intArray0 = NewSubject.getVersionNo("$S3P&,*V+D&1{%U6");
        assertEquals(4, intArray0.length);
        assertNotNull(intArray0);
        assertEquals(new int[] {6, 0, 0, 0}, intArray0);
    }

    @Test(timeout = 4000)
    public void test52() throws Throwable {
        int int0 = NewSubject.extractIntInStr("");
        assertEquals(0, int0);
    }

    @Test(timeout = 4000)
    public void test53() throws Throwable {
        int int0 = NewSubject.extractIntInStr((String) null);
        assertEquals(0, int0);
    }

    @Test(timeout = 4000)
    public void test54() throws Throwable {
        char[] charArray0 = new char[5];
        charArray0[1] = 'A';
    }

```

```

        String string0 = NewSubject.parseToken("Aug", charArray0);
        assertEquals("", string0);
        assertEquals(5, charArray0.length);
        assertNotNull(string0);
        assertEquals(new char[] {'\u0000', 'A', '\u0000', '\u0000', '\u0000'},
charArray0);
    }

    @Test(timeout = 4000)
    public void test55() throws Throwable {
        char[] charArray0 = new char[6];
        String string0 = NewSubject.parseToken("(invalid)", charArray0);
        assertEquals("(invalid)", string0);
        assertEquals(6, charArray0.length);
        assertNotNull(string0);
        assertEquals(new char[] {'\u0000', '\u0000', '\u0000', '\u0000', '\u0000',
'\u0000'}, charArray0);
    }

    @Test(timeout = 4000)
    public void test56() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("(invalid)", "Oct");
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test57() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("N", "Jun");
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test58() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("'Y~", (String) null);
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test59() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("$S3P&,*V+D&1{%U6",
"$S3P&,*V+D&1{%U6");
        assertTrue(boolean0);
    }

    @Test(timeout = 4000)
    public void test60() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase((String) null, "Apr");
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test61() throws Throwable {
        NewSubject newSubject0 = new NewSubject();
        assertNotNull(newSubject0);
    }
}

```

=== Your previous implementation (round 2) ===

```
package comp5111.assignment.cut;
```

```
/**
```

```
 * Task 4 reduced CUT.
```

```

*
* Ten public-static methods selected from {@link Subject}, with their
* documented contracts and one private helper (isOneOf). The patched
* (correct) implementations from Task 2 are used so that EvoSuite's
* generated tests pin down the *intended* behaviour, against which the
* LLM's regenerated code is judged.
*
* Every signature is identical to the corresponding method in Subject.
*/
public class NewSubject {

    // ---- 1 ----
    public static boolean startsWithIgnoreCase(String str, String prefix) {
        if (str == null || prefix == null) {
            return false;
        }
        return str.toLowerCase().startsWith(prefix.toLowerCase());
    }

    // ---- 2 ----
    public static String parseToken(final String str, final char[] terminators) {
        if (str == null || terminators == null) {
            return str;
        }
        for (int i = 0; i < str.length(); i++) {
            for (char terminator : terminators) {
                if (str.charAt(i) == terminator) {
                    return str.substring(0, i);
                }
            }
        }
        return str;
    }

    private static boolean isOneOf(char ch, final char[] chararray) {
        for (char c : chararray) {
            if (c == ch) {
                return true;
            }
        }
        return false;
    }

    // ---- 3 ----
    public static int extractIntInStr(String str) {
        if (str == null || str.isEmpty()) {
            return 0;
        }
        StringBuilder digits = new StringBuilder();
        for (int i = str.length() - 1; i >= 0; i--) {
            char ch = str.charAt(i);
            if (Character.isDigit(ch)) {
                digits.append(ch);
            } else if (digits.length() > 0) {
                break;
            }
        }
        return digits.length() > 0 ? Integer.parseInt(digits.reverse().toString()) : 0;
    }

    // ---- 4 ----
    public static int[] getVersionNo(final String versionString) {
        if (versionString == null || versionString.isEmpty()) {
            return null;
        }
    }

```

```

    }
    String[] parts = versionString.split("\\.");
    if (parts.length > 4) {
        return null;
    }
    int[] versionNumbers = new int[4];
    for (int i = 0; i < parts.length; i++) {
        try {
            versionNumbers[i] = Integer.parseInt(parts[i]);
        } catch (NumberFormatException e) {
            return null;
        }
    }
    return versionNumbers;
}

// ---- 5 ----
public static String padLeft(String str, short length, char padChar) {
    if (str == null) {
        str = "";
    }
    if (str.length() >= length) {
        return str;
    }
    StringBuilder padded = new StringBuilder();
    for (int i = str.length(); i < length; i++) {
        padded.append(padChar);
    }
    padded.append(str);
    return padded.toString();
}

// ---- 6 ----
public static boolean judgeLeapYear(int year) {
    if (year < 0) {
        return false;
    }
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

// ---- 7 ----
public static int calcDaysInMonth(int year, int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    switch (month) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            return 31;
        case 4: case 6: case 9: case 11:
            return 30;
        case 2:
            return judgeLeapYear(year) ? 29 : 28;
        default:
            return -1;
    }
}

// ---- 8 ----
public static int getQuarter(int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    return (month - 1) / 3 + 1;
}

```

```

// ---- 9 ----
public static int monAbbr2month(String abbr) {
    if (abbr == null || abbr.length() != 3) {
        return -1;
    }
    switch (abbr) {
        case "Jan": return 1;
        case "Feb": return 2;
        case "Mar": return 3;
        case "Apr": return 4;
        case "May": return 5;
        case "Jun": return 6;
        case "Jul": return 7;
        case "Aug": return 8;
        case "Sep": return 9;
        case "Oct": return 10;
        case "Nov": return 11;
        case "Dec": return 12;
        default: return -1;
    }
}

// ---- 10 ----
public static String month2MonAbbr(int month) {
    switch (month) {
        case 1: return "Jan";
        case 2: return "Feb";
        case 3: return "Mar";
        case 4: return "Apr";
        case 5: return "May";
        case 6: return "Jun";
        case 7: return "Jul";
        case 8: return "Aug";
        case 9: return "Sep";
        case 10: return "Oct";
        case 11: return "Nov";
        case 12: return "Dec";
        default: return "(invalid)";
    }
}
}

```

=== Test failures observed for that implementation ===

2 of 62 tests failed.

- test51(comp5111.assignment.cut.NewSubject_ESTest) :: null
- test43(comp5111.assignment.cut.NewSubject_ESTest) :: expected:<29> but was:<28>

Produce a corrected NewSubject.java that fixes ALL of the failures above without breaking previously passing tests.

Raw LLM response (5110 chars)

```

```java
package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
 *

```

```

* Ten public-static methods selected from {@link Subject}, with their
* documented contracts and one private helper (isOneOf). The patched
* (correct) implementations from Task 2 are used so that EvoSuite's
* generated tests pin down the *intended* behaviour, against which the
* LLM's regenerated code is judged.
*
* Every signature is identical to the corresponding method in Subject.
*/
public class NewSubject {

 // ---- 1 ----
 public static boolean startsWithIgnoreCase(String str, String prefix) {
 if (str == null || prefix == null) {
 return false;
 }
 return str.toLowerCase().startsWith(prefix.toLowerCase());
 }

 // ---- 2 ----
 public static String parseToken(final String str, final char[] terminators) {
 if (str == null || terminators == null) {
 return str;
 }
 for (int i = 0; i < str.length(); i++) {
 for (char terminator : terminators) {
 if (str.charAt(i) == terminator) {
 return str.substring(0, i);
 }
 }
 }
 return str;
 }

 private static boolean isOneOf(char ch, final char[] chararray) {
 for (char c : chararray) {
 if (c == ch) {
 return true;
 }
 }
 return false;
 }

 // ---- 3 ----
 public static int extractIntInStr(String str) {
 if (str == null || str.isEmpty()) {
 return 0;
 }
 StringBuilder digits = new StringBuilder();
 for (int i = str.length() - 1; i >= 0; i--) {
 char ch = str.charAt(i);
 if (Character.isDigit(ch)) {
 digits.append(ch);
 } else if (digits.length() > 0) {
 break;
 }
 }
 return digits.length() > 0 ? Integer.parseInt(digits.reverse().toString()) : 0;
 }

 // ---- 4 ----
 public static int[] getVersionNo(final String versionString) {
 if (versionString == null || versionString.isEmpty()) {
 return null;
 }
 }

```

```

 String[] parts = versionString.split("\\.");
 if (parts.length > 4) {
 return null;
 }
 int[] versionNumbers = new int[4];
 for (int i = 0; i < parts.length; i++) {
 try {
 versionNumbers[i] = Integer.parseInt(parts[i]);
 } catch (NumberFormatException e) {
 return null;
 }
 }
 return versionNumbers;
 }

 // ---- 5 ----
 public static String padLeft(String str, short length, char padChar) {
 if (str == null) {
 str = "";
 }
 if (str.length() >= length) {
 return str;
 }
 StringBuilder padded = new StringBuilder();
 for (int i = str.length(); i < length; i++) {
 padded.append(padChar);
 }
 padded.append(str);
 return padded.toString();
 }

 // ---- 6 ----
 public static boolean judgeLeapYear(int year) {
 if (year < 0) {
 return false;
 }
 return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
 }

 // ---- 7 ----
 public static int calcDaysInMonth(int year, int month) {
 if (month < 1 || month > 12) {
 return -1;
 }
 switch (month) {
 case 1: case 3: case 5: case 7: case 8: case 10: case 12:
 return 31;
 case 4: case 6: case 9: case 11:
 return 30;
 case 2:
 return judgeLeapYear(year) ? 29 : 28;
 default:
 return -1;
 }
 }

 // ---- 8 ----
 public static int getQuarter(int month) {
 if (month < 1 || month > 12) {
 return -1;
 }
 return (month - 1) / 3 + 1;
 }

```



```

// ---- 9 ----
public static int monAbbr2month(String abbr) {
 if (abbr == null || abbr.length() != 3) {
 return -1;
 }
 switch (abbr) {
 case "Jan": return 1;
 case "Feb": return 2;
 case "Mar": return 3;
 case "Apr": return 4;
 case "May": return 5;
 case "Jun": return 6;
 case "Jul": return 7;
 case "Aug": return 8;
 case "Sep": return 9;
 case "Oct": return 10;
 case "Nov": return 11;
 case "Dec": return 12;
 default: return -1;
 }
}

// ---- 10 ----
public static String month2MonAbbr(int month) {
 switch (month) {
 case 1: return "Jan";
 case 2: return "Feb";
 case 3: return "Mar";
 case 4: return "Apr";
 case 5: return "May";
 case 6: return "Jun";
 case 7: return "Jul";
 case 8: return "Aug";
 case 9: return "Sep";
 case 10: return "Oct";
 case 11: return "Nov";
 case 12: return "Dec";
 default: return "(invalid)";
 }
}
}

```

### Extracted NewSubject.java for this round

```

```java
package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf). The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's
 * generated tests pin down the *intended* behaviour, against which the
 * LLM's regenerated code is judged.
 *
 * Every signature is identical to the corresponding method in Subject.
 */
public class NewSubject {

    // ---- 1 ----
    public static boolean startsWithIgnoreCase(String str, String prefix) {

```

```

        if (str == null || prefix == null) {
            return false;
        }
        return str.toLowerCase().startsWith(prefix.toLowerCase());
    }

    // ---- 2 ----
    public static String parseToken(final String str, final char[] terminators) {
        if (str == null || terminators == null) {
            return str;
        }
        for (int i = 0; i < str.length(); i++) {
            for (char terminator : terminators) {
                if (str.charAt(i) == terminator) {
                    return str.substring(0, i);
                }
            }
        }
        return str;
    }

    private static boolean isOneOf(char ch, final char[] chararray) {
        for (char c : chararray) {
            if (c == ch) {
                return true;
            }
        }
        return false;
    }

    // ---- 3 ----
    public static int extractIntInStr(String str) {
        if (str == null || str.isEmpty()) {
            return 0;
        }
        StringBuilder digits = new StringBuilder();
        for (int i = str.length() - 1; i >= 0; i--) {
            char ch = str.charAt(i);
            if (Character.isDigit(ch)) {
                digits.append(ch);
            } else if (digits.length() > 0) {
                break;
            }
        }
        return digits.length() > 0 ? Integer.parseInt(digits.reverse().toString()) : 0;
    }

    // ---- 4 ----
    public static int[] getVersionNo(final String versionString) {
        if (versionString == null || versionString.isEmpty()) {
            return null;
        }
        String[] parts = versionString.split("\\.");
        if (parts.length > 4) {
            return null;
        }
        int[] versionNumbers = new int[4];
        for (int i = 0; i < parts.length; i++) {
            try {
                versionNumbers[i] = Integer.parseInt(parts[i]);
            } catch (NumberFormatException e) {
                return null;
            }
        }
    }

```

```

        return versionNumbers;
    }

    // ---- 5 ----
    public static String padLeft(String str, short length, char padChar) {
        if (str == null) {
            str = "";
        }
        if (str.length() >= length) {
            return str;
        }
        StringBuilder padded = new StringBuilder();
        for (int i = str.length(); i < length; i++) {
            padded.append(padChar);
        }
        padded.append(str);
        return padded.toString();
    }

    // ---- 6 ----
    public static boolean judgeLeapYear(int year) {
        if (year < 0) {
            return false;
        }
        return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
    }

    // ---- 7 ----
    public static int calcDaysInMonth(int year, int month) {
        if (month < 1 || month > 12) {
            return -1;
        }
        switch (month) {
            case 1: case 3: case 5: case 7: case 8: case 10: case 12:
                return 31;
            case 4: case 6: case 9: case 11:
                return 30;
            case 2:
                return judgeLeapYear(year) ? 29 : 28;
            default:
                return -1;
        }
    }

    // ---- 8 ----
    public static int getQuarter(int month) {
        if (month < 1 || month > 12) {
            return -1;
        }
        return (month - 1) / 3 + 1;
    }

    // ---- 9 ----
    public static int monAbbr2month(String abbr) {
        if (abbr == null || abbr.length() != 3) {
            return -1;
        }
        switch (abbr) {
            case "Jan": return 1;
            case "Feb": return 2;
            case "Mar": return 3;
            case "Apr": return 4;
            case "May": return 5;
            case "Jun": return 6;
        }
    }

```

```

        case "Jul": return 7;
        case "Aug": return 8;
        case "Sep": return 9;
        case "Oct": return 10;
        case "Nov": return 11;
        case "Dec": return 12;
        default: return -1;
    }
}

// ---- 10 ----
public static String month2MonAbbr(int month) {
    switch (month) {
        case 1: return "Jan";
        case 2: return "Feb";
        case 3: return "Mar";
        case 4: return "Apr";
        case 5: return "May";
        case 6: return "Jun";
        case 7: return "Jul";
        case 8: return "Aug";
        case 9: return "Sep";
        case 10: return "Oct";
        case 11: return "Nov";
        case 12: return "Dec";
        default: return "(invalid)";
    }
}
}

```

Round 4

Test outcome

```

{
  "compile_ok": true,
  "run_count": 62,
  "fail_count": 2,
  "failures": [
    "test51(comp5111.assignment.cut.NewSubject_ESTest) :: null",
    "test43(comp5111.assignment.cut.NewSubject_ESTest) :: expected:<29> but was:<28>"
  ],
  "error": "",
  "raw_stdout": "10:33:57.856 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject_ESTest\n10:33:57.857 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject_ESTest'.\n10:33:57.884 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject_ESTest_scaffolding\n10:33:57.884 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject_ESTest_scaffolding'.\n10:33:57.886 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject_ESTest_scaffolding\n10:33:57.886 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject_ESTest\n10:33:58.061 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread: \"Reference
Handler\"\n10:33:58.061 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Finalizer\"\n10:33:58.061 [main] DEBUG

```

```

org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread: \"Signal
Dispatcher\"\\n10:33:58.061 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Java2D Disposer\"\\n10:33:58.061 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread:
\"main\"\\n10:33:58.061 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Common-Cleaner\"\\n10:33:58.079 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject\\n10:33:58.079 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject'.\\n10:33:58.084 [main] INFO
org.evosuite.runtime.instrumentation.MethodCallReplacementClassAdapter - Adding mock
interface to class comp5111/assignment/cut/NewSubject\\n10:33:58.152 [main] INFO
org.evosuite.runtime.instrumentation.CreateClassResetClassAdapter - Creating brand-new
static initializer in class comp5111/assignment/cut/NewSubject\\n10:33:58.153 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject\\n10:33:58.164 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:58.170 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:58.174 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:58.180 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:58.181 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:58.193 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:58.194 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:58.195 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:58.197 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:33:58.198 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new
thread\\nRUN_COUNT=62\\nFAIL_COUNT=2\\nFAILURE>>>test51(comp5111.assignment.cut.NewSubject_EST
null<<<FAILURE\\nFAILURE>>>test43(comp5111.assignment.cut.NewSubject_ESTest) ::
expected:<29> but was:<28><<<FAILURE\\n\",
  \"raw_stderr\": \"WARNING: An illegal reflective access operation has occurred\\nWARNING:
Illegal reflective access by org.evosuite.runtime.GuiSupport (file:/home/zimo/playground/
COMP5111-2026Spring-Assignments-Students/lib/evosuite-1.2.0.jar) to field
java.awt.GraphicsEnvironment.headless\\nWARNING: Please consider reporting this to the
maintainers of org.evosuite.runtime.GuiSupport\\nWARNING: Use --illegal-access=warn to
enable warnings of further illegal reflective access operations\\nWARNING: All illegal
access operations will be denied in a future release\\n\"
}

```

Prompt sent to LLM (20628 chars)

You are a Java engineer. Implement the class `comp5111.assignment.cut.NewSubject` so that every JUnit test in the provided test suite passes.

Rules:

- Output exactly ONE fenced ``java code block containing the FULL contents of NewSubject.java (package, imports if any, the entire class).
- Do NOT modify the test suite. Do NOT add main(). Keep the package and class name unchanged. Method signatures shown below are mandatory.
- Output nothing outside the single code block.

=== Method signatures + Javadoc (mandatory shape) ===

```
package comp5111.assignment.cut;
```

```
/**
```

```
 * Task 4 reduced CUT.
```

```
 *
```

```
 * Ten public-static methods selected from {@link Subject}, with their
```

```
 * documented contracts and one private helper (isOneOf). The patched
```

```
 * (correct) implementations from Task 2 are used so that EvoSuite's
```

```
 * generated tests pin down the *intended* behaviour, against which the
```

```

* LLM's regenerated code is judged.
*
* Every signature is identical to the corresponding method in Subject.
*/
public class NewSubject {

    // ---- 1 ----
    /**
     * Returns true if str starts with prefix ignoring case, else false.
     * Returns false on any null argument or if str is shorter than prefix.
     */
    public static boolean startsWithIgnoreCase(String str, String prefix);

    // ---- 2 ----
    /**
     * Returns the prefix of str up to (but not including) the first
     * character that appears in {@code terminators}. If no terminator
     * is found, the whole string is returned.
     */
    public static String parseToken(final String str, final char[] terminators);

    private static boolean isOneOf(char ch, final char[] chararray);

    // ---- 3 ----
    /**
     * Returns the LAST contiguous run of digits in str interpreted as
     * an int. Empty/null/no-digit inputs return 0.
     * Examples: "1234" -> 1234, "a" -> 0, "1234a123" -> 123.
     */
    public static int extractIntInStr(String str);

    // ---- 4 ----
    /**
     * Parses a version string of form a, a.b, a.b.c, or a.b.c.d into a 4-int
     * array. Trailing components default to 0. More than 4 dot-segments or
     * malformed (consecutive dots) produces null.
     */
    public static int[] getVersionNo(final String versionString);

    // ---- 5 ----
    /**
     * Pads str on the left with padChar so the resulting length is at least
     * length. Null str is treated as "". If str is already long enough it is
     * returned unchanged.
     */
    public static String padLeft(String str, short length, char padChar);

    // ---- 6 ----
    /** Returns true iff year is a leap year (Gregorian rule). */
    public static boolean judgeLeapYear(int year);

    // ---- 7 ----
    /**
     * Returns the number of days in the given month (1-12) of the given
     * year. Returns -1 if month is out of range. February uses leap-year
     * rules.
     */
    public static int calcDaysInMonth(int year, int month);

    // ---- 8 ----
    /** Returns the calendar quarter (1-4) for month (1-12), or -1 if invalid. */
    public static int getQuarter(int month);

    // ---- 9 ----

```

```

/**
 * Converts a 3-char month abbreviation (case-sensitive, e.g. "Jan",
 * "Sep") into its month number (1-12). Returns -1 on null, wrong
 * length, or unknown abbreviation.
 */
public static int monAbbr2month(String abbr);

// ---- 10 ----
/**
 * Inverse of monAbbr2month: returns "Jan".. "Dec" for month 1..12, and
 * "(invalid)" otherwise.
 */
public static String month2MonAbbr(int month);
}

```

=== EvoSuite-generated test suite (passes against the ground truth) ===

```

/*
 * This file was automatically generated by EvoSuite
 * Thu May 07 11:32:05 GMT 2026
 */

package comp5111.assignment.cut;

import org.junit.Test;
import static org.junit.Assert.*;
import comp5111.assignment.cut.NewSubject;
import org.evosuite.runtime.EvoRunner;
import org.evosuite.runtime.EvoRunnerParameters;
import org.junit.runner.RunWith;

@RunWith(EvoRunner.class) @EvoRunnerParameters(mockJVMNonDeterminism = true, useVFS =
true, useVNET = true, resetStaticState = true, separateClassLoader = true)
public class NewSubject_ESTest extends NewSubject_ESTest_scaffolding {

    @Test(timeout = 4000)
    public void test00() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(1493);
        assertEquals("(invalid)", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test01() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(12);
        assertEquals("Dec", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test02() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(11);
        assertEquals("Nov", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test03() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(10);
        assertEquals("Oct", string0);
        assertNotNull(string0);
    }
}

```

```
@Test(timeout = 4000)
public void test04() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(9);
    assertEquals("Sep", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test05() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(8);
    assertEquals("Aug", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test06() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(7);
    assertEquals("Jul", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test07() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(6);
    assertEquals("Jun", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test08() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(5);
    assertEquals("May", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test09() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(4);
    assertEquals("Apr", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test10() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(3);
    assertEquals("Mar", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test11() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(2);
    assertEquals("Feb", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test12() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(1);
    assertEquals("Jan", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
```



```
public void test13() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Dec");
    assertEquals(12, int0);
}

@Test(timeout = 4000)
public void test14() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Nov");
    assertEquals(11, int0);
}

@Test(timeout = 4000)
public void test15() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Oct");
    assertEquals(10, int0);
}

@Test(timeout = 4000)
public void test16() throws Throwable {
    int int0 = NewSubject.monAbbr2month("'Y~");
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test17() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Aug");
    assertEquals(8, int0);
}

@Test(timeout = 4000)
public void test18() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Jul");
    assertEquals(7, int0);
}

@Test(timeout = 4000)
public void test19() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Jun");
    assertEquals(6, int0);
}

@Test(timeout = 4000)
public void test20() throws Throwable {
    int int0 = NewSubject.monAbbr2month("May");
    assertEquals(5, int0);
}

@Test(timeout = 4000)
public void test21() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Apr");
    assertEquals(4, int0);
}

@Test(timeout = 4000)
public void test22() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Mar");
    assertEquals(3, int0);
}

@Test(timeout = 4000)
public void test23() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Sep");
    assertEquals(9, int0);
}
```

```

@Test(timeout = 4000)
public void test24() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Jan");
    assertEquals(1, int0);
}

@Test(timeout = 4000)
public void test25() throws Throwable {
    int int0 = NewSubject.monAbbr2month("(invalid)");
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test26() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Feb");
    assertEquals(2, int0);
}

@Test(timeout = 4000)
public void test27() throws Throwable {
    int int0 = NewSubject.monAbbr2month((String) null);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test28() throws Throwable {
    int int0 = NewSubject.getQuarter(11);
    assertEquals(4, int0);
}

@Test(timeout = 4000)
public void test29() throws Throwable {
    int int0 = NewSubject.getQuarter(4);
    assertEquals(2, int0);
}

@Test(timeout = 4000)
public void test30() throws Throwable {
    int int0 = NewSubject.getQuarter((short)8);
    assertEquals(3, int0);
}

@Test(timeout = 4000)
public void test31() throws Throwable {
    int int0 = NewSubject.getQuarter(946);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test32() throws Throwable {
    int int0 = NewSubject.getQuarter(2);
    assertEquals(1, int0);
}

@Test(timeout = 4000)
public void test33() throws Throwable {
    int int0 = NewSubject.getQuarter((-1460));
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test34() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth(11, 11);

```

```

        assertEquals(30, int0);
    }

    @Test(timeout = 4000)
    public void test35() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth((-1), 9);
        assertEquals(30, int0);
    }

    @Test(timeout = 4000)
    public void test36() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth(0, 6);
        assertEquals(30, int0);
    }

    @Test(timeout = 4000)
    public void test37() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth(2, 4);
        assertEquals(30, int0);
    }

    @Test(timeout = 4000)
    public void test38() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth((-1), 2);
        assertEquals(28, int0);
    }

    @Test(timeout = 4000)
    public void test39() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth(0, 3);
        assertEquals(31, int0);
    }

    @Test(timeout = 4000)
    public void test40() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth(1, 1726);
        assertEquals((-1), int0);
    }

    @Test(timeout = 4000)
    public void test41() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth((-3324), (-1570));
        assertEquals((-1), int0);
    }

    @Test(timeout = 4000)
    public void test42() throws Throwable {
        boolean boolean0 = NewSubject.judgeLeapYear(3300);
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test43() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth((-3324), 2);
        assertEquals(29, int0);
    }

    @Test(timeout = 4000)
    public void test44() throws Throwable {
        boolean boolean0 = NewSubject.judgeLeapYear(0);
        assertTrue(boolean0);
    }

    @Test(timeout = 4000)

```

```

public void test45() throws Throwable {
    String string0 = NewSubject.padLeft("Jun", (short)1741, 'G');
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test46() throws Throwable {
    String string0 = NewSubject.padLeft((String) null, (short) (-1), '(');
    assertEquals("", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test47() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo(".:r~w'iW");
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test48() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo("oZ.>B6oZ.>B6oZ.>B6oZ.>B6");
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test49() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo("");
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test50() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo((String) null);
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test51() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo("$S3P&,*V+D&1{%U6");
    assertEquals(4, intArray0.length);
    assertNotNull(intArray0);
    assertEquals(new int[] {6, 0, 0, 0}, intArray0);
}

@Test(timeout = 4000)
public void test52() throws Throwable {
    int int0 = NewSubject.extractIntInStr("");
    assertEquals(0, int0);
}

@Test(timeout = 4000)
public void test53() throws Throwable {
    int int0 = NewSubject.extractIntInStr((String) null);
    assertEquals(0, int0);
}

@Test(timeout = 4000)
public void test54() throws Throwable {
    char[] charArray0 = new char[5];
    charArray0[1] = 'A';
    String string0 = NewSubject.parseToken("Aug", charArray0);
    assertEquals("", string0);
    assertEquals(5, charArray0.length);
    assertNotNull(string0);
}

```

```

        assertEquals(new char[] {'\u0000', 'A', '\u0000', '\u0000', '\u0000'},
charArray0);
    }

    @Test(timeout = 4000)
    public void test55() throws Throwable {
        char[] charArray0 = new char[6];
        String string0 = NewSubject.parseToken("(invalid)", charArray0);
        assertEquals("(invalid)", string0);
        assertEquals(6, charArray0.length);
        assertNotNull(string0);
        assertEquals(new char[] {'\u0000', '\u0000', '\u0000', '\u0000', '\u0000',
'\u0000'}, charArray0);
    }

    @Test(timeout = 4000)
    public void test56() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("(invalid)", "Oct");
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test57() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("N", "Jun");
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test58() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("'Y~", (String) null);
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test59() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("$S3P&,*V+D&l{%U6",
"$S3P&,*V+D&l{%U6");
        assertTrue(boolean0);
    }

    @Test(timeout = 4000)
    public void test60() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase((String) null, "Apr");
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test61() throws Throwable {
        NewSubject newSubject0 = new NewSubject();
        assertNotNull(newSubject0);
    }
}

```

=== Your previous implementation (round 3) ===

```
package comp5111.assignment.cut;
```

```

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf). The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's

```

```

* generated tests pin down the *intended* behaviour, against which the
* LLM's regenerated code is judged.
*
* Every signature is identical to the corresponding method in Subject.
*/
public class NewSubject {

    // ---- 1 ----
    public static boolean startsWithIgnoreCase(String str, String prefix) {
        if (str == null || prefix == null) {
            return false;
        }
        return str.toLowerCase().startsWith(prefix.toLowerCase());
    }

    // ---- 2 ----
    public static String parseToken(final String str, final char[] terminators) {
        if (str == null || terminators == null) {
            return str;
        }
        for (int i = 0; i < str.length(); i++) {
            for (char terminator : terminators) {
                if (str.charAt(i) == terminator) {
                    return str.substring(0, i);
                }
            }
        }
        return str;
    }

    private static boolean isOneOf(char ch, final char[] chararray) {
        for (char c : chararray) {
            if (c == ch) {
                return true;
            }
        }
        return false;
    }

    // ---- 3 ----
    public static int extractIntInStr(String str) {
        if (str == null || str.isEmpty()) {
            return 0;
        }
        StringBuilder digits = new StringBuilder();
        for (int i = str.length() - 1; i >= 0; i--) {
            char ch = str.charAt(i);
            if (Character.isDigit(ch)) {
                digits.append(ch);
            } else if (digits.length() > 0) {
                break;
            }
        }
        return digits.length() > 0 ? Integer.parseInt(digits.reverse().toString()) : 0;
    }

    // ---- 4 ----
    public static int[] getVersionNo(final String versionString) {
        if (versionString == null || versionString.isEmpty()) {
            return null;
        }
        String[] parts = versionString.split("\\.");
        if (parts.length > 4) {
            return null;
        }
    }
}

```

```

    }
    int[] versionNumbers = new int[4];
    for (int i = 0; i < parts.length; i++) {
        try {
            versionNumbers[i] = Integer.parseInt(parts[i]);
        } catch (NumberFormatException e) {
            return null;
        }
    }
    return versionNumbers;
}

// ---- 5 ----
public static String padLeft(String str, short length, char padChar) {
    if (str == null) {
        str = "";
    }
    if (str.length() >= length) {
        return str;
    }
    StringBuilder padded = new StringBuilder();
    for (int i = str.length(); i < length; i++) {
        padded.append(padChar);
    }
    padded.append(str);
    return padded.toString();
}

// ---- 6 ----
public static boolean judgeLeapYear(int year) {
    if (year < 0) {
        return false;
    }
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

// ---- 7 ----
public static int calcDaysInMonth(int year, int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    switch (month) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            return 31;
        case 4: case 6: case 9: case 11:
            return 30;
        case 2:
            return judgeLeapYear(year) ? 29 : 28;
        default:
            return -1;
    }
}

// ---- 8 ----
public static int getQuarter(int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    return (month - 1) / 3 + 1;
}

// ---- 9 ----
public static int monAbbr2month(String abbr) {
    if (abbr == null || abbr.length() != 3) {

```

```

        return -1;
    }
    switch (abbr) {
        case "Jan": return 1;
        case "Feb": return 2;
        case "Mar": return 3;
        case "Apr": return 4;
        case "May": return 5;
        case "Jun": return 6;
        case "Jul": return 7;
        case "Aug": return 8;
        case "Sep": return 9;
        case "Oct": return 10;
        case "Nov": return 11;
        case "Dec": return 12;
        default: return -1;
    }
}

// ---- 10 ----
public static String month2MonAbbr(int month) {
    switch (month) {
        case 1: return "Jan";
        case 2: return "Feb";
        case 3: return "Mar";
        case 4: return "Apr";
        case 5: return "May";
        case 6: return "Jun";
        case 7: return "Jul";
        case 8: return "Aug";
        case 9: return "Sep";
        case 10: return "Oct";
        case 11: return "Nov";
        case 12: return "Dec";
        default: return "(invalid)";
    }
}
}

```

=== Test failures observed for that implementation ===

2 of 62 tests failed.

- test51(comp5111.assignment.cut.NewSubject_ESTest) :: null
- test43(comp5111.assignment.cut.NewSubject_ESTest) :: expected:<29> but was:<28>

Produce a corrected NewSubject.java that fixes ALL of the failures above without breaking previously passing tests.

Raw LLM response (5110 chars)

```

```java
package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf). The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's
 * generated tests pin down the *intended* behaviour, against which the

```



```

* LLM's regenerated code is judged.
*
* Every signature is identical to the corresponding method in Subject.
*/
public class NewSubject {

 // ---- 1 ----
 public static boolean startsWithIgnoreCase(String str, String prefix) {
 if (str == null || prefix == null) {
 return false;
 }
 return str.toLowerCase().startsWith(prefix.toLowerCase());
 }

 // ---- 2 ----
 public static String parseToken(final String str, final char[] terminators) {
 if (str == null || terminators == null) {
 return str;
 }
 for (int i = 0; i < str.length(); i++) {
 for (char terminator : terminators) {
 if (str.charAt(i) == terminator) {
 return str.substring(0, i);
 }
 }
 }
 return str;
 }

 private static boolean isOneOf(char ch, final char[] chararray) {
 for (char c : chararray) {
 if (c == ch) {
 return true;
 }
 }
 return false;
 }

 // ---- 3 ----
 public static int extractIntInStr(String str) {
 if (str == null || str.isEmpty()) {
 return 0;
 }
 StringBuilder digits = new StringBuilder();
 for (int i = str.length() - 1; i >= 0; i--) {
 char ch = str.charAt(i);
 if (Character.isDigit(ch)) {
 digits.append(ch);
 } else if (digits.length() > 0) {
 break;
 }
 }
 return digits.length() > 0 ? Integer.parseInt(digits.reverse().toString()) : 0;
 }

 // ---- 4 ----
 public static int[] getVersionNo(final String versionString) {
 if (versionString == null || versionString.isEmpty()) {
 return null;
 }
 String[] parts = versionString.split("\\.");
 if (parts.length > 4) {
 return null;
 }
 }
}

```

```

 int[] versionNumbers = new int[4];
 for (int i = 0; i < parts.length; i++) {
 try {
 versionNumbers[i] = Integer.parseInt(parts[i]);
 } catch (NumberFormatException e) {
 return null;
 }
 }
 return versionNumbers;
 }
}

// ---- 5 ----
public static String padLeft(String str, short length, char padChar) {
 if (str == null) {
 str = "";
 }
 if (str.length() >= length) {
 return str;
 }
 StringBuilder padded = new StringBuilder();
 for (int i = str.length(); i < length; i++) {
 padded.append(padChar);
 }
 padded.append(str);
 return padded.toString();
}

// ---- 6 ----
public static boolean judgeLeapYear(int year) {
 if (year < 0) {
 return false;
 }
 return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

// ---- 7 ----
public static int calcDaysInMonth(int year, int month) {
 if (month < 1 || month > 12) {
 return -1;
 }
 switch (month) {
 case 1: case 3: case 5: case 7: case 8: case 10: case 12:
 return 31;
 case 4: case 6: case 9: case 11:
 return 30;
 case 2:
 return judgeLeapYear(year) ? 29 : 28;
 default:
 return -1;
 }
}

// ---- 8 ----
public static int getQuarter(int month) {
 if (month < 1 || month > 12) {
 return -1;
 }
 return (month - 1) / 3 + 1;
}

// ---- 9 ----
public static int monAbbr2month(String abbr) {
 if (abbr == null || abbr.length() != 3) {
 return -1;
 }
}

```

```

 }
 switch (abbr) {
 case "Jan": return 1;
 case "Feb": return 2;
 case "Mar": return 3;
 case "Apr": return 4;
 case "May": return 5;
 case "Jun": return 6;
 case "Jul": return 7;
 case "Aug": return 8;
 case "Sep": return 9;
 case "Oct": return 10;
 case "Nov": return 11;
 case "Dec": return 12;
 default: return -1;
 }
}

// ---- 10 ----
public static String month2MonAbbr(int month) {
 switch (month) {
 case 1: return "Jan";
 case 2: return "Feb";
 case 3: return "Mar";
 case 4: return "Apr";
 case 5: return "May";
 case 6: return "Jun";
 case 7: return "Jul";
 case 8: return "Aug";
 case 9: return "Sep";
 case 10: return "Oct";
 case 11: return "Nov";
 case 12: return "Dec";
 default: return "(invalid)";
 }
}
}
}

```

### Extracted NewSubject.java for this round

```

```java
package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf). The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's
 * generated tests pin down the intended behaviour, against which the
 * LLM's regenerated code is judged.
 *
 * Every signature is identical to the corresponding method in Subject.
 */
public class NewSubject {

    // ---- 1 ----
    public static boolean startsWithIgnoreCase(String str, String prefix) {
        if (str == null || prefix == null) {
            return false;
        }
        return str.toLowerCase().startsWith(prefix.toLowerCase());
    }
}

```

```

}

// ---- 2 ----
public static String parseToken(final String str, final char[] terminators) {
    if (str == null || terminators == null) {
        return str;
    }
    for (int i = 0; i < str.length(); i++) {
        for (char terminator : terminators) {
            if (str.charAt(i) == terminator) {
                return str.substring(0, i);
            }
        }
    }
    return str;
}

private static boolean isOneOf(char ch, final char[] chararray) {
    for (char c : chararray) {
        if (c == ch) {
            return true;
        }
    }
    return false;
}

// ---- 3 ----
public static int extractIntInStr(String str) {
    if (str == null || str.isEmpty()) {
        return 0;
    }
    StringBuilder digits = new StringBuilder();
    for (int i = str.length() - 1; i >= 0; i--) {
        char ch = str.charAt(i);
        if (Character.isDigit(ch)) {
            digits.append(ch);
        } else if (digits.length() > 0) {
            break;
        }
    }
    return digits.length() > 0 ? Integer.parseInt(digits.reverse().toString()) : 0;
}

// ---- 4 ----
public static int[] getVersionNo(final String versionString) {
    if (versionString == null || versionString.isEmpty()) {
        return null;
    }
    String[] parts = versionString.split("\\.");
    if (parts.length > 4) {
        return null;
    }
    int[] versionNumbers = new int[4];
    for (int i = 0; i < parts.length; i++) {
        try {
            versionNumbers[i] = Integer.parseInt(parts[i]);
        } catch (NumberFormatException e) {
            return null;
        }
    }
    return versionNumbers;
}

// ---- 5 ----

```

```

public static String padLeft(String str, short length, char padChar) {
    if (str == null) {
        str = "";
    }
    if (str.length() >= length) {
        return str;
    }
    StringBuilder padded = new StringBuilder();
    for (int i = str.length(); i < length; i++) {
        padded.append(padChar);
    }
    padded.append(str);
    return padded.toString();
}

// ---- 6 ----
public static boolean judgeLeapYear(int year) {
    if (year < 0) {
        return false;
    }
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

// ---- 7 ----
public static int calcDaysInMonth(int year, int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    switch (month) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            return 31;
        case 4: case 6: case 9: case 11:
            return 30;
        case 2:
            return judgeLeapYear(year) ? 29 : 28;
        default:
            return -1;
    }
}

// ---- 8 ----
public static int getQuarter(int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    return (month - 1) / 3 + 1;
}

// ---- 9 ----
public static int monAbbr2month(String abbr) {
    if (abbr == null || abbr.length() != 3) {
        return -1;
    }
    switch (abbr) {
        case "Jan": return 1;
        case "Feb": return 2;
        case "Mar": return 3;
        case "Apr": return 4;
        case "May": return 5;
        case "Jun": return 6;
        case "Jul": return 7;
        case "Aug": return 8;
        case "Sep": return 9;
        case "Oct": return 10;
    }
}

```

```

        case "Nov": return 11;
        case "Dec": return 12;
        default: return -1;
    }
}

// ---- 10 ----
public static String month2MonAbbr(int month) {
    switch (month) {
        case 1: return "Jan";
        case 2: return "Feb";
        case 3: return "Mar";
        case 4: return "Apr";
        case 5: return "May";
        case 6: return "Jun";
        case 7: return "Jul";
        case 8: return "Aug";
        case 9: return "Sep";
        case 10: return "Oct";
        case 11: return "Nov";
        case 12: return "Dec";
        default: return "(invalid)";
    }
}
}

```

Round 5

Test outcome

```

{
  "compile_ok": true,
  "run_count": 62,
  "fail_count": 2,
  "failures": [
    "test51(comp5111.assignment.cut.NewSubject_ESTest) :: null",
    "test43(comp5111.assignment.cut.NewSubject_ESTest) :: expected:<29> but was:<28>"
  ],
  "error": "",
  "raw_stdout": "10:34:13.895 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject_ESTest\n10:34:13.896 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject_ESTest'.\n10:34:13.923 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject_ESTest_scaffolding\n10:34:13.924 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject_ESTest_scaffolding'.\n10:34:13.926 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject_ESTest_scaffolding\n10:34:13.926 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject_ESTest\n10:34:14.098 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread: \"Reference
Handler\"\n10:34:14.098 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Finalizer\"\n10:34:14.098 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread: \"Signal
Dispatcher\"\n10:34:14.098 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Java2D Disposer\"\n10:34:14.098 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread:

```

```

\"main\"\\n10:34:14.098 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Common-Cleaner\"\\n10:34:14.116 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject\\n10:34:14.116 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject'.\\n10:34:14.121 [main] INFO
org.evosuite.runtime.instrumentation.MethodCallReplacementClassAdapter - Adding mock
interface to class comp5111/assignment/cut/NewSubject\\n10:34:14.191 [main] INFO
org.evosuite.runtime.instrumentation.CreateClassResetClassAdapter - Creating brand-new
static initializer in class comp5111/assignment/cut/NewSubject\\n10:34:14.192 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject\\n10:34:14.204 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:34:14.208 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:34:14.209 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:34:14.218 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:34:14.227 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:34:14.228 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:34:14.229 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:34:14.231 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:34:14.232 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:34:14.233 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:34:14.234 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:34:14.234 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:34:14.237 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\\n10:34:14.237 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new
thread\\nRUN_COUNT=62\\nFAIL_COUNT=2\\nFAILURE>>>test51(comp5111.assignment.cut.NewSubject_EST
null<<<FAILURE\\nFAILURE>>>test43(comp5111.assignment.cut.NewSubject_ESTest) ::
expected:<29> but was:<28><<<FAILURE\\n\",
  \"raw_stderr\": \"WARNING: An illegal reflective access operation has occurred\\nWARNING:
Illegal reflective access by org.evosuite.runtime.GuiSupport (file:/home/zimo/playground/
COMP5111-2026Spring-Assignments-Students/lib/evosuite-1.2.0.jar) to field
java.awt.GraphicsEnvironment.headless\\nWARNING: Please consider reporting this to the
maintainers of org.evosuite.runtime.GuiSupport\\nWARNING: Use --illegal-access=warn to
enable warnings of further illegal reflective access operations\\nWARNING: All illegal
access operations will be denied in a future release\\n\"
}

```

Prompt sent to LLM (20628 chars)

You are a Java engineer. Implement the class `comp5111.assignment.cut.NewSubject` so that every JUnit test in the provided test suite passes.

Rules:

- Output exactly ONE fenced ``java code block containing the FULL contents of NewSubject.java (package, imports if any, the entire class).
- Do NOT modify the test suite. Do NOT add main(). Keep the package and class name unchanged. Method signatures shown below are mandatory.
- Output nothing outside the single code block.

=== Method signatures + Javadoc (mandatory shape) ===

```
package comp5111.assignment.cut;
```

```
/**
```

```
 * Task 4 reduced CUT.
```

```
 *
```

```
 * Ten public-static methods selected from {@link Subject}, with their
```

```
 * documented contracts and one private helper (isOneOf). The patched
```

```
 * (correct) implementations from Task 2 are used so that EvoSuite's
```

```
 * generated tests pin down the *intended* behaviour, against which the
```

```

* LLM's regenerated code is judged.
*
* Every signature is identical to the corresponding method in Subject.
*/
public class NewSubject {

    // ---- 1 ----
    /**
     * Returns true if str starts with prefix ignoring case, else false.
     * Returns false on any null argument or if str is shorter than prefix.
     */
    public static boolean startsWithIgnoreCase(String str, String prefix);

    // ---- 2 ----
    /**
     * Returns the prefix of str up to (but not including) the first
     * character that appears in {@code terminators}. If no terminator
     * is found, the whole string is returned.
     */
    public static String parseToken(final String str, final char[] terminators);

    private static boolean isOneOf(char ch, final char[] chararray);

    // ---- 3 ----
    /**
     * Returns the LAST contiguous run of digits in str interpreted as
     * an int. Empty/null/no-digit inputs return 0.
     * Examples: "1234" -> 1234, "a" -> 0, "1234a123" -> 123.
     */
    public static int extractIntInStr(String str);

    // ---- 4 ----
    /**
     * Parses a version string of form a, a.b, a.b.c, or a.b.c.d into a 4-int
     * array. Trailing components default to 0. More than 4 dot-segments or
     * malformed (consecutive dots) produces null.
     */
    public static int[] getVersionNo(final String versionString);

    // ---- 5 ----
    /**
     * Pads str on the left with padChar so the resulting length is at least
     * length. Null str is treated as "". If str is already long enough it is
     * returned unchanged.
     */
    public static String padLeft(String str, short length, char padChar);

    // ---- 6 ----
    /** Returns true iff year is a leap year (Gregorian rule). */
    public static boolean judgeLeapYear(int year);

    // ---- 7 ----
    /**
     * Returns the number of days in the given month (1-12) of the given
     * year. Returns -1 if month is out of range. February uses leap-year
     * rules.
     */
    public static int calcDaysInMonth(int year, int month);

    // ---- 8 ----
    /** Returns the calendar quarter (1-4) for month (1-12), or -1 if invalid. */
    public static int getQuarter(int month);

    // ---- 9 ----

```



```

/**
 * Converts a 3-char month abbreviation (case-sensitive, e.g. "Jan",
 * "Sep") into its month number (1-12). Returns -1 on null, wrong
 * length, or unknown abbreviation.
 */
public static int monAbbr2month(String abbr);

// ---- 10 ----
/**
 * Inverse of monAbbr2month: returns "Jan".. "Dec" for month 1..12, and
 * "(invalid)" otherwise.
 */
public static String month2MonAbbr(int month);
}

```

=== EvoSuite-generated test suite (passes against the ground truth) ===

```

/*
 * This file was automatically generated by EvoSuite
 * Thu May 07 11:32:05 GMT 2026
 */

package comp5111.assignment.cut;

import org.junit.Test;
import static org.junit.Assert.*;
import comp5111.assignment.cut.NewSubject;
import org.evosuite.runtime.EvoRunner;
import org.evosuite.runtime.EvoRunnerParameters;
import org.junit.runner.RunWith;

@RunWith(EvoRunner.class) @EvoRunnerParameters(mockJVMNonDeterminism = true, useVFS =
true, useVNET = true, resetStaticState = true, separateClassLoader = true)
public class NewSubject_ESTest extends NewSubject_ESTest_scaffolding {

    @Test(timeout = 4000)
    public void test00() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(1493);
        assertEquals("(invalid)", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test01() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(12);
        assertEquals("Dec", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test02() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(11);
        assertEquals("Nov", string0);
        assertNotNull(string0);
    }

    @Test(timeout = 4000)
    public void test03() throws Throwable {
        String string0 = NewSubject.month2MonAbbr(10);
        assertEquals("Oct", string0);
        assertNotNull(string0);
    }
}

```

```
@Test(timeout = 4000)
public void test04() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(9);
    assertEquals("Sep", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test05() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(8);
    assertEquals("Aug", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test06() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(7);
    assertEquals("Jul", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test07() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(6);
    assertEquals("Jun", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test08() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(5);
    assertEquals("May", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test09() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(4);
    assertEquals("Apr", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test10() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(3);
    assertEquals("Mar", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test11() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(2);
    assertEquals("Feb", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test12() throws Throwable {
    String string0 = NewSubject.month2MonAbbr(1);
    assertEquals("Jan", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
```

```
public void test13() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Dec");
    assertEquals(12, int0);
}

@Test(timeout = 4000)
public void test14() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Nov");
    assertEquals(11, int0);
}

@Test(timeout = 4000)
public void test15() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Oct");
    assertEquals(10, int0);
}

@Test(timeout = 4000)
public void test16() throws Throwable {
    int int0 = NewSubject.monAbbr2month("'Y~");
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test17() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Aug");
    assertEquals(8, int0);
}

@Test(timeout = 4000)
public void test18() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Jul");
    assertEquals(7, int0);
}

@Test(timeout = 4000)
public void test19() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Jun");
    assertEquals(6, int0);
}

@Test(timeout = 4000)
public void test20() throws Throwable {
    int int0 = NewSubject.monAbbr2month("May");
    assertEquals(5, int0);
}

@Test(timeout = 4000)
public void test21() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Apr");
    assertEquals(4, int0);
}

@Test(timeout = 4000)
public void test22() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Mar");
    assertEquals(3, int0);
}

@Test(timeout = 4000)
public void test23() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Sep");
    assertEquals(9, int0);
}
```

```

@Test(timeout = 4000)
public void test24() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Jan");
    assertEquals(1, int0);
}

@Test(timeout = 4000)
public void test25() throws Throwable {
    int int0 = NewSubject.monAbbr2month("(invalid)");
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test26() throws Throwable {
    int int0 = NewSubject.monAbbr2month("Feb");
    assertEquals(2, int0);
}

@Test(timeout = 4000)
public void test27() throws Throwable {
    int int0 = NewSubject.monAbbr2month((String) null);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test28() throws Throwable {
    int int0 = NewSubject.getQuarter(11);
    assertEquals(4, int0);
}

@Test(timeout = 4000)
public void test29() throws Throwable {
    int int0 = NewSubject.getQuarter(4);
    assertEquals(2, int0);
}

@Test(timeout = 4000)
public void test30() throws Throwable {
    int int0 = NewSubject.getQuarter((short)8);
    assertEquals(3, int0);
}

@Test(timeout = 4000)
public void test31() throws Throwable {
    int int0 = NewSubject.getQuarter(946);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test32() throws Throwable {
    int int0 = NewSubject.getQuarter(2);
    assertEquals(1, int0);
}

@Test(timeout = 4000)
public void test33() throws Throwable {
    int int0 = NewSubject.getQuarter((-1460));
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test34() throws Throwable {
    int int0 = NewSubject.calcDaysInMonth(11, 11);

```

```

        assertEquals(30, int0);
    }

    @Test(timeout = 4000)
    public void test35() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth((-1), 9);
        assertEquals(30, int0);
    }

    @Test(timeout = 4000)
    public void test36() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth(0, 6);
        assertEquals(30, int0);
    }

    @Test(timeout = 4000)
    public void test37() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth(2, 4);
        assertEquals(30, int0);
    }

    @Test(timeout = 4000)
    public void test38() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth((-1), 2);
        assertEquals(28, int0);
    }

    @Test(timeout = 4000)
    public void test39() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth(0, 3);
        assertEquals(31, int0);
    }

    @Test(timeout = 4000)
    public void test40() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth(1, 1726);
        assertEquals((-1), int0);
    }

    @Test(timeout = 4000)
    public void test41() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth((-3324), (-1570));
        assertEquals((-1), int0);
    }

    @Test(timeout = 4000)
    public void test42() throws Throwable {
        boolean boolean0 = NewSubject.judgeLeapYear(3300);
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test43() throws Throwable {
        int int0 = NewSubject.calcDaysInMonth((-3324), 2);
        assertEquals(29, int0);
    }

    @Test(timeout = 4000)
    public void test44() throws Throwable {
        boolean boolean0 = NewSubject.judgeLeapYear(0);
        assertTrue(boolean0);
    }

    @Test(timeout = 4000)

```

```

public void test45() throws Throwable {
    String string0 = NewSubject.padLeft("Jun", (short)1741, 'G');
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test46() throws Throwable {
    String string0 = NewSubject.padLeft((String) null, (short) (-1), '(');
    assertEquals("", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test47() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo(".:r~w'iW");
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test48() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo("oZ.>B6oZ.>B6oZ.>B6oZ.>B6");
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test49() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo("");
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test50() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo((String) null);
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test51() throws Throwable {
    int[] intArray0 = NewSubject.getVersionNo("$S3P&,*V+D&1{%U6");
    assertEquals(4, intArray0.length);
    assertNotNull(intArray0);
    assertEquals(new int[] {6, 0, 0, 0}, intArray0);
}

@Test(timeout = 4000)
public void test52() throws Throwable {
    int int0 = NewSubject.extractIntInStr("");
    assertEquals(0, int0);
}

@Test(timeout = 4000)
public void test53() throws Throwable {
    int int0 = NewSubject.extractIntInStr((String) null);
    assertEquals(0, int0);
}

@Test(timeout = 4000)
public void test54() throws Throwable {
    char[] charArray0 = new char[5];
    charArray0[1] = 'A';
    String string0 = NewSubject.parseToken("Aug", charArray0);
    assertEquals("", string0);
    assertEquals(5, charArray0.length);
    assertNotNull(string0);
}

```

```

        assertEquals(new char[] {'\u0000', 'A', '\u0000', '\u0000', '\u0000'},
charArray0);
    }

    @Test(timeout = 4000)
    public void test55() throws Throwable {
        char[] charArray0 = new char[6];
        String string0 = NewSubject.parseToken("(invalid)", charArray0);
        assertEquals("(invalid)", string0);
        assertEquals(6, charArray0.length);
        assertNotNull(string0);
        assertEquals(new char[] {'\u0000', '\u0000', '\u0000', '\u0000', '\u0000',
'\u0000'}, charArray0);
    }

    @Test(timeout = 4000)
    public void test56() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("(invalid)", "Oct");
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test57() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("N", "Jun");
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test58() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("'Y~", (String) null);
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test59() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase("$S3P&,*V+D&l{%U6",
"$S3P&,*V+D&l{%U6");
        assertTrue(boolean0);
    }

    @Test(timeout = 4000)
    public void test60() throws Throwable {
        boolean boolean0 = NewSubject.startsWithIgnoreCase((String) null, "Apr");
        assertFalse(boolean0);
    }

    @Test(timeout = 4000)
    public void test61() throws Throwable {
        NewSubject newSubject0 = new NewSubject();
        assertNotNull(newSubject0);
    }
}

```

=== Your previous implementation (round 4) ===

```
package comp5111.assignment.cut;
```

```
/**
```

```
 * Task 4 reduced CUT.
```

```
 *
```

```

 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf). The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's

```

```

* generated tests pin down the *intended* behaviour, against which the
* LLM's regenerated code is judged.
*
* Every signature is identical to the corresponding method in Subject.
*/
public class NewSubject {

    // ---- 1 ----
    public static boolean startsWithIgnoreCase(String str, String prefix) {
        if (str == null || prefix == null) {
            return false;
        }
        return str.toLowerCase().startsWith(prefix.toLowerCase());
    }

    // ---- 2 ----
    public static String parseToken(final String str, final char[] terminators) {
        if (str == null || terminators == null) {
            return str;
        }
        for (int i = 0; i < str.length(); i++) {
            for (char terminator : terminators) {
                if (str.charAt(i) == terminator) {
                    return str.substring(0, i);
                }
            }
        }
        return str;
    }

    private static boolean isOneOf(char ch, final char[] chararray) {
        for (char c : chararray) {
            if (c == ch) {
                return true;
            }
        }
        return false;
    }

    // ---- 3 ----
    public static int extractIntInStr(String str) {
        if (str == null || str.isEmpty()) {
            return 0;
        }
        StringBuilder digits = new StringBuilder();
        for (int i = str.length() - 1; i >= 0; i--) {
            char ch = str.charAt(i);
            if (Character.isDigit(ch)) {
                digits.append(ch);
            } else if (digits.length() > 0) {
                break;
            }
        }
        return digits.length() > 0 ? Integer.parseInt(digits.reverse().toString()) : 0;
    }

    // ---- 4 ----
    public static int[] getVersionNo(final String versionString) {
        if (versionString == null || versionString.isEmpty()) {
            return null;
        }
        String[] parts = versionString.split("\\.");
        if (parts.length > 4) {
            return null;
        }
    }
}

```



```

    }
    int[] versionNumbers = new int[4];
    for (int i = 0; i < parts.length; i++) {
        try {
            versionNumbers[i] = Integer.parseInt(parts[i]);
        } catch (NumberFormatException e) {
            return null;
        }
    }
    return versionNumbers;
}

// ---- 5 ----
public static String padLeft(String str, short length, char padChar) {
    if (str == null) {
        str = "";
    }
    if (str.length() >= length) {
        return str;
    }
    StringBuilder padded = new StringBuilder();
    for (int i = str.length(); i < length; i++) {
        padded.append(padChar);
    }
    padded.append(str);
    return padded.toString();
}

// ---- 6 ----
public static boolean judgeLeapYear(int year) {
    if (year < 0) {
        return false;
    }
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

// ---- 7 ----
public static int calcDaysInMonth(int year, int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    switch (month) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            return 31;
        case 4: case 6: case 9: case 11:
            return 30;
        case 2:
            return judgeLeapYear(year) ? 29 : 28;
        default:
            return -1;
    }
}

// ---- 8 ----
public static int getQuarter(int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    return (month - 1) / 3 + 1;
}

// ---- 9 ----
public static int monAbbr2month(String abbr) {
    if (abbr == null || abbr.length() != 3) {

```

```

        return -1;
    }
    switch (abbr) {
        case "Jan": return 1;
        case "Feb": return 2;
        case "Mar": return 3;
        case "Apr": return 4;
        case "May": return 5;
        case "Jun": return 6;
        case "Jul": return 7;
        case "Aug": return 8;
        case "Sep": return 9;
        case "Oct": return 10;
        case "Nov": return 11;
        case "Dec": return 12;
        default: return -1;
    }
}

// ---- 10 ----
public static String month2MonAbbr(int month) {
    switch (month) {
        case 1: return "Jan";
        case 2: return "Feb";
        case 3: return "Mar";
        case 4: return "Apr";
        case 5: return "May";
        case 6: return "Jun";
        case 7: return "Jul";
        case 8: return "Aug";
        case 9: return "Sep";
        case 10: return "Oct";
        case 11: return "Nov";
        case 12: return "Dec";
        default: return "(invalid)";
    }
}
}

```

=== Test failures observed for that implementation ===

2 of 62 tests failed.

- test51(comp5111.assignment.cut.NewSubject_ESTest) :: null
- test43(comp5111.assignment.cut.NewSubject_ESTest) :: expected:<29> but was:<28>

Produce a corrected NewSubject.java that fixes ALL of the failures above without breaking previously passing tests.

Raw LLM response (5110 chars)

```

```java
package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf). The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's
 * generated tests pin down the *intended* behaviour, against which the

```

```

* LLM's regenerated code is judged.
*
* Every signature is identical to the corresponding method in Subject.
*/
public class NewSubject {

 // ---- 1 ----
 public static boolean startsWithIgnoreCase(String str, String prefix) {
 if (str == null || prefix == null) {
 return false;
 }
 return str.toLowerCase().startsWith(prefix.toLowerCase());
 }

 // ---- 2 ----
 public static String parseToken(final String str, final char[] terminators) {
 if (str == null || terminators == null) {
 return str;
 }
 for (int i = 0; i < str.length(); i++) {
 for (char terminator : terminators) {
 if (str.charAt(i) == terminator) {
 return str.substring(0, i);
 }
 }
 }
 return str;
 }

 private static boolean isOneOf(char ch, final char[] chararray) {
 for (char c : chararray) {
 if (c == ch) {
 return true;
 }
 }
 return false;
 }

 // ---- 3 ----
 public static int extractIntInStr(String str) {
 if (str == null || str.isEmpty()) {
 return 0;
 }
 StringBuilder digits = new StringBuilder();
 for (int i = str.length() - 1; i >= 0; i--) {
 char ch = str.charAt(i);
 if (Character.isDigit(ch)) {
 digits.append(ch);
 } else if (digits.length() > 0) {
 break;
 }
 }
 return digits.length() > 0 ? Integer.parseInt(digits.reverse().toString()) : 0;
 }

 // ---- 4 ----
 public static int[] getVersionNo(final String versionString) {
 if (versionString == null || versionString.isEmpty()) {
 return null;
 }
 String[] parts = versionString.split("\\.");
 if (parts.length > 4) {
 return null;
 }
 }
}

```

```

 int[] versionNumbers = new int[4];
 for (int i = 0; i < parts.length; i++) {
 try {
 versionNumbers[i] = Integer.parseInt(parts[i]);
 } catch (NumberFormatException e) {
 return null;
 }
 }
 return versionNumbers;
 }
}

// ---- 5 ----
public static String padLeft(String str, short length, char padChar) {
 if (str == null) {
 str = "";
 }
 if (str.length() >= length) {
 return str;
 }
 StringBuilder padded = new StringBuilder();
 for (int i = str.length(); i < length; i++) {
 padded.append(padChar);
 }
 padded.append(str);
 return padded.toString();
}

// ---- 6 ----
public static boolean judgeLeapYear(int year) {
 if (year < 0) {
 return false;
 }
 return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

// ---- 7 ----
public static int calcDaysInMonth(int year, int month) {
 if (month < 1 || month > 12) {
 return -1;
 }
 switch (month) {
 case 1: case 3: case 5: case 7: case 8: case 10: case 12:
 return 31;
 case 4: case 6: case 9: case 11:
 return 30;
 case 2:
 return judgeLeapYear(year) ? 29 : 28;
 default:
 return -1;
 }
}

// ---- 8 ----
public static int getQuarter(int month) {
 if (month < 1 || month > 12) {
 return -1;
 }
 return (month - 1) / 3 + 1;
}

// ---- 9 ----
public static int monAbbr2month(String abbr) {
 if (abbr == null || abbr.length() != 3) {
 return -1;
 }
}

```

```

 }
 switch (abbr) {
 case "Jan": return 1;
 case "Feb": return 2;
 case "Mar": return 3;
 case "Apr": return 4;
 case "May": return 5;
 case "Jun": return 6;
 case "Jul": return 7;
 case "Aug": return 8;
 case "Sep": return 9;
 case "Oct": return 10;
 case "Nov": return 11;
 case "Dec": return 12;
 default: return -1;
 }
}

// ---- 10 ----
public static String month2MonAbbr(int month) {
 switch (month) {
 case 1: return "Jan";
 case 2: return "Feb";
 case 3: return "Mar";
 case 4: return "Apr";
 case 5: return "May";
 case 6: return "Jun";
 case 7: return "Jul";
 case 8: return "Aug";
 case 9: return "Sep";
 case 10: return "Oct";
 case 11: return "Nov";
 case 12: return "Dec";
 default: return "(invalid)";
 }
}
}
}

```

### Extracted NewSubject.java for this round

```

```java
package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf). The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's
 * generated tests pin down the *intended* behaviour, against which the
 * LLM's regenerated code is judged.
 *
 * Every signature is identical to the corresponding method in Subject.
 */
public class NewSubject {

    // ---- 1 ----
    public static boolean startsWithIgnoreCase(String str, String prefix) {
        if (str == null || prefix == null) {
            return false;
        }
        return str.toLowerCase().startsWith(prefix.toLowerCase());
    }
}

```

```

}

// ---- 2 ----
public static String parseToken(final String str, final char[] terminators) {
    if (str == null || terminators == null) {
        return str;
    }
    for (int i = 0; i < str.length(); i++) {
        for (char terminator : terminators) {
            if (str.charAt(i) == terminator) {
                return str.substring(0, i);
            }
        }
    }
    return str;
}

private static boolean isOneOf(char ch, final char[] chararray) {
    for (char c : chararray) {
        if (c == ch) {
            return true;
        }
    }
    return false;
}

// ---- 3 ----
public static int extractIntInStr(String str) {
    if (str == null || str.isEmpty()) {
        return 0;
    }
    StringBuilder digits = new StringBuilder();
    for (int i = str.length() - 1; i >= 0; i--) {
        char ch = str.charAt(i);
        if (Character.isDigit(ch)) {
            digits.append(ch);
        } else if (digits.length() > 0) {
            break;
        }
    }
    return digits.length() > 0 ? Integer.parseInt(digits.reverse().toString()) : 0;
}

// ---- 4 ----
public static int[] getVersionNo(final String versionString) {
    if (versionString == null || versionString.isEmpty()) {
        return null;
    }
    String[] parts = versionString.split("\\.");
    if (parts.length > 4) {
        return null;
    }
    int[] versionNumbers = new int[4];
    for (int i = 0; i < parts.length; i++) {
        try {
            versionNumbers[i] = Integer.parseInt(parts[i]);
        } catch (NumberFormatException e) {
            return null;
        }
    }
    return versionNumbers;
}

// ---- 5 ----

```

```

public static String padLeft(String str, short length, char padChar) {
    if (str == null) {
        str = "";
    }
    if (str.length() >= length) {
        return str;
    }
    StringBuilder padded = new StringBuilder();
    for (int i = str.length(); i < length; i++) {
        padded.append(padChar);
    }
    padded.append(str);
    return padded.toString();
}

// ---- 6 ----
public static boolean judgeLeapYear(int year) {
    if (year < 0) {
        return false;
    }
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

// ---- 7 ----
public static int calcDaysInMonth(int year, int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    switch (month) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            return 31;
        case 4: case 6: case 9: case 11:
            return 30;
        case 2:
            return judgeLeapYear(year) ? 29 : 28;
        default:
            return -1;
    }
}

// ---- 8 ----
public static int getQuarter(int month) {
    if (month < 1 || month > 12) {
        return -1;
    }
    return (month - 1) / 3 + 1;
}

// ---- 9 ----
public static int monAbbr2month(String abbr) {
    if (abbr == null || abbr.length() != 3) {
        return -1;
    }
    switch (abbr) {
        case "Jan": return 1;
        case "Feb": return 2;
        case "Mar": return 3;
        case "Apr": return 4;
        case "May": return 5;
        case "Jun": return 6;
        case "Jul": return 7;
        case "Aug": return 8;
        case "Sep": return 9;
        case "Oct": return 10;
    }
}

```

```
        case "Nov": return 11;
        case "Dec": return 12;
        default: return -1;
    }
}

// ---- 10 ----
public static String month2MonAbbr(int month) {
    switch (month) {
        case 1: return "Jan";
        case 2: return "Feb";
        case 3: return "Mar";
        case 4: return "Apr";
        case 5: return "May";
        case 6: return "Jun";
        case 7: return "Jul";
        case 8: return "Aug";
        case 9: return "Sep";
        case 10: return "Oct";
        case 11: return "Nov";
        case 12: return "Dec";
        default: return "(invalid)";
    }
}
}
```
